Q-Pathformer Tiered Model Signal Exchange and Controlled Maturity Routing for AI Training Systems

Interaction-Role Signal Exchange Across Model Classes with Governance-Constrained Dataset Formation

Compatible with Access-PoD Governance and Q-Pathformer Execution Environments

Document Control Information
Document Identifier: APOD-TR-008
Title: Q-Pathformer Tiered Model Signal Exchange and Controlled Maturity Routing for AI Training Systems
Subtitle: Interaction-Role Signal Exchange Across Model Classes with Governance-Constrained Dataset Formation
Edition: First Edition
Date of Issue: 18th April 2026
Status: Public Review Draft (Standards Exploration)
Author: Anthony Siljeg
Affiliation: Alnotrea Labs / Policy of Developments - Access PoD *Initiative.*

Document Scope

This technical report defines an execution-layer framework for tiered model signal exchange, controlled maturity routing, and governance-constrained dataset formation in AI training systems.

The report explains how signal interactions across model classes can be captured, structured, routed, packaged, frozen, and prepared for training within lifecycle-controlled environments.

It establishes governance-aware execution semantics, maturity-routing boundaries, and dataset formation controls, but does not prescribe a single machine learning architecture, model family, training pipeline, or deployment environment.

Execution systems such as Q-Pathformer may consume lifecycle-governed datasets produced under this framework, but such systems do not define governance authority within the Access-PoD model.

This document extends the governance framework established in APOD-TR-002 (Version A), the architecture illustration described in APOD-TR-003 (Version B), the onboarding discipline defined in APOD-TR-004 (Version C), and the dataset lifecycle and passport model defined in APOD-TR-007. It introduces the execution-layer routing model for governed signal exchange, controlled maturity progression, and dataset formation in AI training environments.

TR-002 → TR-003 → TR-004 → TR-007 → TR-008
Concept → Architecture → Onboarding → Dataset Lifecycle → Execution Routing

Abstract

This technical report extends the governed dataset lifecycle framework defined in APOD-TR-007 by introducing a tier-aware signal interaction model for AI training environments operating across multiple model classes.

The report defines a structured execution model for controlled signal exchange between model tiers. In the present Q-Pathformer release, runtime signal generation uses a 20B-class model (gpt-oss:20b), and, after explicit promotion into STATE_1, training preparation uses a 1.1B local LoRA base (TinyLlama/TinyLlama-1.1B-Chat-v1.0). New source signals and newly onboarded external artifacts enter under STATE_3 containment by default; controlled candidate packages may be handled in STATE_2 for review, while STATE_1 is reserved for promoted datasets prepared for governed training use. The active Q-Pathformer GitHub release branches are the reference baseline (reference_sparse_e08 agents released) and the enterprise target (grouped_sparse_e64 agents released). Governance schemas distinguish enterprise routing ranges (12B–14B) from optional strategic ranges (24B–30B), while strategic profiles such as fused_sparse_e144 remain internal architectural references for future API-mediated signal access rather than required release components.

Signal interactions are represented through a structured interaction-role model in which signals generated by one system may be interpreted, filtered, and transformed by another into governed dataset artifacts. These roles are contextual to each interaction and do not imply hierarchy, authority, or a trust gradient between model tiers.

All cross-tier signals are treated as provisional artifacts and remain subject to lifecycle containment as defined in APOD-TR-007. Signals and newly onboarded external artifacts are assigned to STATE_3 by default and may support runtime reasoning, synthetic dataset generation, or exploratory analysis. STATE_2 provides the controlled validation and convergence layer for candidate datasets, while progression to STATE_1 requires explicit validation, authorization filtering, issuance of a governed dataset passport, and promotion under governed review.

The report introduces a signal maturity classification model to support diagnostic evaluation of signal quality and convergence. Maturity levels are advisory only and do not imply correctness, adequacy, or authority.

The execution model preserves strict separation between governance authority and system behavior. Governance authority, authorization, and assurance semantics remain defined externally within the Access-PoD framework. Execution environments may implement signal capture, routing, and dataset formation mechanisms without inheriting governance authority.

In the present release, this framework is realized across three governed dataset formation paths: a TR corpus and reference path, an Access-Mode ID_2 to ID_3 canonical lifecycle path, and an external or custom dataset adapter path. The 1.1B training base is used only

after a dataset has been promoted into STATE_1 for controlled training preparation within the end-to-end state flow, while traceability, containment, and governance discipline are preserved throughout dataset evolution.

Current Release Reference Note

1. Three-Path Execution Reference Model

This release records the current TR-008 reference model as a governed execution environment expressed through three distinct dataset-formation pathways within Q-Pathformer DEV.

The release aligns the execution-layer routing described in APOD-TR-008 with the dataset lifecycle defined in APOD-TR-007 while preserving the principle that governance authority remains external to the execution system.

The three execution paths in the current release are:

- Path 1 - TR corpus and reference path
- Path 2 - Access-Mode ID_2 to ID_3 canonical lifecycle path
- Path 3 - external or custom dataset adapter path
- Taken together, these pathways show how source references, runtime signals, and external bundles may enter a common governed lifecycle under default STATE_3 containment, while prepared candidate packages may proceed to STATE_2 for controlled review, without collapsing authority, approval, or trust into execution logic.
- No source artifact enters directly at STATE_1. STATE_1 is reached only through explicit validation, promotion, handoff, freeze, and registry controls where applicable.

All paths remain non-authoritative and operate under fail-closed validation, deterministic lineage handling, and explicit freeze boundaries where applicable.

---

2. Execution Scope

The current release establishes three reference dataset pathways:

- Path 1 - transformation of the technical report (TR) corpus and reference materials into governed datasets under lifecycle containment, with explicit review and promotion records before STATE_1 training preparation

- Path 2 - governance-aware handling of Access-Mode artifacts from STATE_3 intake, through STATE_2 review, toward controlled STATE_1 handoff, freeze, and registry preparation
- Path 3 - onboarding of external or custom artifacts through adapter normalization, provenance preservation, and contained STATE_3 intake, with later optional handoff into the governed lifecycle
- cross-path lineage propagation and dataset passport continuity
- deterministic freeze and archive execution with cryptographic integrity, where applicable

The execution environment operates under:

- ID_2 reasoning posture for signal capture, dataset structuring, candidate formation, and controlled runtime interpretation
- ID_3 reasoning posture for freeze evidence, inspection preparation, and canonical archive packaging

No execution path introduces governance authority, certification, compliance assertions, or automatic approval.

New source data does not enter directly at STATE_1; STATE_1 is reserved for explicitly promoted datasets prepared for training export, handoff, freeze, registry use, and LoRA execution.

3. Verified Properties

3.1 Lineage Propagation

Lineage was propagated across all lifecycle stages without regeneration or substitution:

```
export lineage.json
→ packaged governance/dataset_lineage.json
→ pre-freeze normalized_governance/dataset_lineage.json
→ freeze (via prepared manifest)
```

Lineage remained:

- traceable
- non-authoritative
- consistent with dataset passport identity

3.2 Dataset Count Integrity

Dataset counts remained consistent across all stages:

- dataset row count
- manifest candidate count

- packaged dataset count
- pre-freeze reconciliation

All counts matched under fail-closed validation.

3.3 Governance Artifact Consistency

The following governance artifacts were preserved and validated:

- dataset passport
- routing record
- authorization report
- lineage record

Pre-freeze normalization ensured:

- governance flags were explicitly set
- lifecycle state alignment was maintained
- containment and non-authoritative constraints were preserved

---

3.4 Canonical Evidence Verification

Canonical inspection artifacts were verified against expected SHA-256 hashes.

- all canonical artifacts matched expected values
- no hash mismatches were detected
- freeze execution completed with zero findings

3.5 Deterministic Freeze Output

Freeze execution produced a structured, inspection-ready artifact set:

```
frozen_package_root/<package-id>--FREEZE-<timestamp>/
├── dataset_package_snapshot/
├── canonical_artifacts/
├── freeze_evidence/
  └── freeze_manifest.json
```

The freeze manifest records:

- source prepared manifest
- prepared freeze input reference
- packaged artifact locations

- canonical artifact verification
- full hash manifest

---

4. Architectural Outcome

This phase establishes a stable execution pattern based on three distinct layers:

4.1 Candidate Layer

- export-ready dataset artifacts
- lineage emitted at source

4.2 Prepared Freeze Input Layer

- prepared freeze input record
- normalized governance artifacts
- prepared freeze manifest defining artifact set

4.3 Frozen Package Output Layer

- sealed ID_3 artifact snapshot
- immutable package and evidence structure
- inspection-ready dataset state

This separation ensures that:

- execution artifacts are not conflated with inspection artifacts
- freeze input is deterministic and explicitly declared
- lineage and governance remain traceable across transitions

---

5. Governance Posture

Outputs produced in this release remain subject to the following constraints:

- Signals do not constitute decisions.
- Datasets remain non-authoritative.
- Freeze does not imply approval or compliance.
- Lifecycle state does not imply correctness or adequacy.
- Authority remains external to execution systems.

The freeze boundary represents:

- an artifact-state transition,
- not a governance decision

6. Alignment with APOD-TR-008

This release provides a reference execution instance of the TR-008 model across three complementary pathways:

- Path 1 - TR corpus and reference-driven dataset formation
- Path 2 - canonical Access-Mode lifecycle routing and controlled maturity progression
- Path 3 - external artifact onboarding and adapter normalization under containment
- cross-path governance-aware packaging, lineage preservation, and controlled freeze boundaries
- signal exchange without transfer of authority between model tiers

The release confirms that the TR-008 execution model can be realized:

- without embedding governance authority,
- while preserving traceability and containment,
- and while maintaining deterministic artifact integrity

---

7. Limitations

This release does not:

- permit new source data to enter directly at STATE_1
- treat execution as the authority for STATE_1 promotion
- treat production training as a governance decision
- assert dataset quality, correctness, compliance, or regulatory sufficiency

All outputs remain:

- non-authoritative within the execution system
- contained in STATE_3 or STATE_2 unless explicitly promoted into STATE_1
- subject to external interpretation, approval, and review

---

8. Phase Status

STATUS: SEALED

This phase demonstrates the release lifecycle in summary:

- new source data enters at STATE_3
- controlled review occurs in STATE_2 where applicable

- STATE_1 is reached only through explicit promotion for training preparation, handoff, freeze, or registry use

The release is:

- internally consistent
- traceable across all stages
- aligned with governance and lifecycle constraints
- It provides a reproducible reference pattern for:
- multi-path execution, dataset evolution, inspection workflows, and standards-aligned development

---

9. Forward Path

Future work may extend this release to include:

- multi-cycle dataset accumulation across the three paths
- stronger STATE_3 to STATE_2 automation and clearer STATE_2 to STATE_1 promotion evidence
- evaluation, drift integration, and branch comparison across released models
- inspection and audit tooling over frozen outputs

All extensions must preserve:

- STATE_3 as the default entry state
- explicit review and promotion before STATE_1
- governance separation, lifecycle containment, and non-authoritative execution posture

Contents

Abstract 2

Current Release Reference Note 3

1. Three-Path Execution Reference Model 3

2. Execution Scope 3

3. Verified Properties 4

3.1 Lineage Propagation 4

3.2 Dataset Count Integrity 4

3.3 Governance Artifact Consistency 5

3.4 Canonical Evidence Verification 5

3.5 Deterministic Freeze Output 5

4. Architectural Outcome 6

4.1 Candidate Layer 6

4.2 Prepared Freeze Input Layer 6

4.3 Frozen Package Output Layer 6

5. Governance Posture 6

6. Alignment with APOD-TR-008 7

7. Limitations 7

8. Phase Status 7

9. Forward Path 8

1. Introduction 16

1.1 Purpose of TR-008 16

1.2 Scope and Applicability 16

1.3 Relationship to Access-PoD and Dataset Lifecycle Framework 17

1.4 Relationship to Working-PoD Artifact Lifecycle 18

1.5 Terminology and Definitions 18

1.6 Document Structure 19

2. Positioning of TR-008 20

2.1 TR-008 as an Execution-Oriented Extension 20

2.2 Separation of Reasoning, Artifact State, and Dataset Lifecycle 20

2.3 Signal Exchange as a Non-Authoritative Interaction Layer 21

2.4 Execution Layer vs Governance Authority 22
2.5 Non-Certifying and Non-Authoritative Posture 23
2.6 Transition to Architectural Definition 24
3. Architectural Overview 24
3.1 Overview of the Tiered Signal Architecture 24
3.2 Tiered Model Interaction Structure 25
3.3 Signal-First System Design 26
3.4 Separation of Runtime, Governance, and Dataset Layers 26
3.5 Signal-to-Dataset Transformation Pipeline 27
3.6 Dataset Packaging and Freeze Integration 28
3.7 Execution-Agnostic Design 28
3.8 Architectural Constraints and Invariants 29
3.9 Transition to Tier Definitions 29
4. Model Tier Definitions 30
4.1 Purpose of Tier Definition 30
4.2 Enterprise-Tier Models 30
4.3 Strategic-Tier Models 31
4.5 Capability vs Authority Separation 32
4.6 Parameter Class vs Deployment Profile 33
4.7 Controlled Cross-Tier Interaction 34
4.8 Prevention of Cross-Tier Authority Transfer 34
4.9 Transition to Signal Structure 35
5. Signal Record and Schema Contract 35
5.1 Purpose of the Signal Record 35
5.2 Signal as the Primary System Artifact 35
5.3 Core Signal Record Structure 36
5.4 Governance Flags and Constraint Fields 37
5.5 Model Invocation and Interaction Metadata 37
5.6 Lifecycle Declaration Within the Signal Record 38
5.7 Signal Maturity as a Diagnostic Classification 38
5.8 Lineage and Traceability Requirements 39

5.9 Non-Authoritative Signal Constraints 39
5.10 Transition to Signal Lifecycle and Routing 40
6. Signal Lifecycle and Dataset State Mapping 40
6.1 Purpose of Lifecycle Mapping 40
6.2 Signal-to-Dataset Transformation Boundary 41
6.3 Default Lifecycle Assignment (STATE_3) 41
6.4 Containment and Non-Authoritative Constraints 42
6.5 Lifecycle States and Dataset Roles 42
6.6 Routing and Lifecycle Transition Logic 43
6.7 Authorization and Filtering Constraints 44
6.8 Signal Convergence (STATE_2) 44
6.9 Dataset Promotion Constraints 45
6.10 Lifecycle Traceability and Lineage 45
6.11 Lifecycle Invariants 46
6.12 Transition to Routing and Control Policies 46
7. Routing and Control Policies 47
7.1 Purpose of Routing and Control 47
7.2 Governance-First Routing Principle 47
7.3 Deterministic Routing Model 48
7.4 Fixed Gate Evaluation Order 48
7.5 Routing Outcomes 50
7.6 Authorization and Validation Separation 51
7.7 Governance Flag Enforcement 51
7.8 Cross-Tier Routing Constraints 52
7.9 Escalation Policy (Enterprise → Strategic) 52
7.10 Routing Logs and Traceability 53
7.11 Failure Handling and Conservative Defaults 54
7.12 Routing Invariants 54
7.13 Transition to Runtime Integration 55
8. Runtime Integration 55
8.1 Purpose of Runtime Integration 55

8.2 Signal Emission from Runtime Systems 56
8.3 Enterprise-Tier Execution as Control Layer 56
8.4 Strategic-Tier Invocation (Optional Layer) 57
8.5 Pipeline Orchestration and Execution Flow 58
8.6 Model Endpoint Abstraction 58
8.7 Local vs Remote Execution 59
8.8 Runtime Configuration and Control Parameters 60
8.9 Failure Handling and Fallback Behavior 60
8.10 Signal Artifact Storage 61
8.11 Runtime Invariants 61
8.12 Transition to Dataset Formation 62
9. Candidate Formation and Fine-Tuning Inputs 62
9.1 Purpose of Candidate Formation 62
9.2 Separation of Signals and Training Data 63
9.3 Candidate Eligibility Criteria 64
9.4 Source Eligibility Constraints 64
9.5 Candidate Dataset Types 65
9.6 Candidate Record Structure 66
9.7 Authorization and Review Requirements 66
9.8 Transformation Discipline 67
9.9 Dataset Candidate Packaging 68
9.10 Fine-Tuning Readiness Gates 68
9.11 Training System Boundary 68
9.12 Traceability and Lineage Requirements 69
9.13 Candidate Formation Invariants 69
9.14 Transition to Evaluation and Control 70
10. Evaluation, Drift, and Revocation 70
10.1 Purpose of Evaluation and Control 70
10.2 Evaluation Posture and Constraints 71
10.3 Baseline Definition and Preservation 71
10.4 Post-Update Evaluation Process 72

10.5 Signal Quality Metrics and Interpretation 73
10.6 Cross-Tier Evaluation 73
10.7 Drift Detection 74
10.8 Drift Thresholds and Interpretation 75
10.9 Revocation Triggers 75
10.10 Revocation and Rollback Actions 75
10.11 Continuous Evaluation Loop 76
10.12 Governance Safeguards 76
10.13 Storage and Traceability 77
10.14 Evaluation Invariants 78
10.15 Transition to Implementation Guidance 78
11. Implementation Guidance (Non-Normative) 78
11.1 Purpose of Implementation Guidance 78
11.2 Execution Environments 79
11.3 Model Access and Registry Patterns 79
11.4 Signal Pipeline Implementation 80
11.5 Dataset Handling and Storage 81
11.6 Fine-Tuning Approaches (Non-Prescriptive) 81
11.7 Hardware and Infrastructure Considerations 82
11.8 Integration with External Tooling 82
11.9 Logging and Observability 83
11.10 Failure Handling and Recovery 84
11.11 Implementation Boundaries 84
11.12 Transition to Governance Alignment 84
12. Dataset Freeze Boundary (ID_3 Packaging and Archive Freeze) 85
12.1 Purpose of the Freeze Boundary 85
12.2 Freeze as an Artifact State Transition 86
12.3 Relationship to Dataset Lifecycle States 86
12.4 Freeze Input Requirements 87
12.5 Frozen Package Structure 88
12.6 Hash Integrity and Verification 89

12.7 Governance Semantics of Freeze 89
12.8 Distinction Between Freeze and Promotion 90
12.9 Freeze Output and Traceability 90
13. Relationship to Working-PoD Lifecycle (TR-006-E.2) 91
13.1 Purpose of Lifecycle Alignment 91
13.2 Distinction Between Reasoning Posture and Artifact State 91
13.3 Working-PoD Lifecycle States 92
13.4 Mapping TR-008 Execution to Lifecycle States 92
13.5 Freeze as the Transition to Inspection Readiness 94
13.6 Prevention of Authority Inflation 94
13.7 Lifecycle Transitions and Explicit Signaling 95
13.8 Relationship Between Lifecycle and Dataset States 95
13.9 Lifecycle Invariants 95
13.10 Transition to Governance Alignment 96
14. Compliance and Governance Alignment 96
14.1 Purpose of Governance Alignment 96
14.2.1 Version A — Governance Authority (Primary Source of Authority) 97
14.2.2 Version B — Architecture and Execution Boundaries 97
14.2.3 Version C — Containment and Onboarding Discipline 98
14.2.4 Version D — Stress and Ethical Constraint 98
14.3 TR-008 Position Within Authority Structure 99
14.4 Dataset Passport and Lifecycle Alignment 99
14.5 Execution vs Governance Authority Separation 100
14.6 Inspection, Freeze, and Evidence Alignment 100
14.7 Standards Positioning 101
14.8 Explicit Non-Claims 101
14.9 Governance Invariants Across Versions 102
15. Conclusion 102
15.1 Summary of Contribution 102
15.2 Integration with Access-PoD Governance Framework 103
15.3 End-to-End Lifecycle Execution Model 103

15.4 Preservation of Governance Boundaries 104
15.5 Execution Without Authority 104
15.6 Role of Freeze in Lifecycle Integrity 105
15.7 Invariants of the TR-008 Model 105
15.8 Standards and Practical Implications 106
15.9 Final Statement 106
Annex B — External Artifact Onboarding and Adapter Ingestion (Informative) 107
References ..... **Error! Bookmark not defined.**

1. Introduction

1.1 Purpose of TR-008

The purpose of this technical report is to extend the governed dataset lifecycle framework defined in APOD-TR-007 by introducing a structured execution model for signal exchange, controlled maturity routing, and governed dataset formation across three complementary paths: a TR corpus and reference path, an Access-Mode canonical lifecycle path, and an external or custom adapter path.

This report defines how signals generated during runtime and exploratory reasoning may be:

- captured,
- structured,
- routed,
- and transformed into governed dataset artifacts,

while preserving:

- provenance,
- authorization scope,
- lifecycle containment,
- and traceability.

The report further introduces a tier-aware interaction model in which AI systems of different capability classes participate in signal generation and interpretation without transferring governance authority or introducing implicit trust.

This document is non-normative and does not define requirements, controls, or conformance criteria. It defines a structured execution model for disciplined dataset formation and lifecycle management.

1.2 Scope and Applicability

This report applies to AI training environments in which:

- Datasets are derived from governance artifacts, operational signals, or reasoning outputs.
- Multiple model classes interact through structured signal exchange.

Dataset evolution must remain governed, traceable, and contained.

The scope of this report includes:

- Path 1 - TR corpus and reference signal formation,
- Path 2 - Access-Mode runtime, review, promotion, handoff, and freeze routing,
- Path 3 - external or custom dataset onboarding and normalization,
- cross-tier signal exchange,
- and lifecycle-controlled dataset packaging, registry preparation, and inspection-oriented freeze boundaries.

The report does not prescribe:

- model architectures,
- training algorithms,
- hardware configurations,
- or deployment environments.

Execution environments such as Q-Pathformer may implement the mechanisms described in this report, but do not define governance authority.

1.3 Relationship to Access-PoD and Dataset Lifecycle Framework

This report operates within the governance model established by the Access-PoD research series.

In particular:

- APOD-TR-006 (Version E) defines reasoning posture control through Access-Mode, separating:
  - learning (ID_1),
  - operational reasoning (ID_2),
  - and inspection reasoning (ID_3).
- APOD-TR-006-E.2 defines the lifecycle of governance artifacts as Working-PoDs, distinguishing:
  - draft,
  - populated,
  - and frozen (inspection-ready) states.
- APOD-TR-007 defines the governed dataset lifecycle, including:
  - dataset passports,
  - lifecycle routing (STATE_3, STATE_2, STATE_1),
  - authorization scope,
  - and controlled promotion.

TR-008 extends these foundations by introducing:

- a signal-first execution model,
- a tier-aware interaction structure across model classes,

- and a governed pathway from signal capture to dataset packaging and freeze.

This report does not modify governance authority or lifecycle semantics defined in earlier publications.
It provides an execution-oriented extension that remains compatible with those frameworks.

1.4 Relationship to Working-PoD Artifact Lifecycle

This report adopts the Working-PoD lifecycle model defined in APOD-TR-006-E.2 as the basis for interpreting artifact state.

Within this model:

- Artifact completeness does not imply authority.
- Reasoning posture does not imply readiness.
- and inspection readiness requires explicit freeze under ID_3 posture.

TR-008 aligns with this model by:

- treating dataset packages as artifact snapshots,
- introducing a pre-freeze preparation layer,
- and defining a freeze boundary that produces inspection-ready, non-authoritative artifact sets.

The freeze boundary described in this report corresponds to the **frozen state** in the Working-PoD lifecycle.

This ensures the following:

- dataset packages assembled under ID_2 remain non-inspectable drafts until explicitly frozen,
- and that inspection readiness is achieved through explicit preparation and declaration, not structural completeness.

1.5 Terminology and Definitions

This report uses terminology consistent with the Access-PoD series.

Key terms include:

Signal
A structured representation of reasoning output or interaction generated during AI-assisted processes.

Signals are treated as provisional artifacts and do not imply correctness, authority, or decision outcomes.

Candidate Dataset
A structured dataset derived from signals that has been filtered, validated, and prepared for inclusion in the governed dataset lifecycle. Candidate datasets remain non-authoritative until explicitly promoted.

Dataset Package
A structured collection of dataset content and associated governance artifacts, including:

- dataset records,
- dataset manifest,
- dataset passport,
- routing record,
- authorization report,
- and lineage record.

Lifecycle State (STATE_1, STATE_2, STATE_3)
A classification of dataset maturity and authorization scope as defined in APOD-TR-007.

Freeze (ID_3 Packaging and Archive Freeze)
An explicit artifact-state transition in which a dataset package is preserved as a stable, inspection-ready snapshot under ID_3 reasoning posture, without implying approval, compliance, or authority.

Canonical Artifact
An inspection-oriented record representing structured reasoning or session context, used to support traceability and interpretation during inspection.

1.6 Document Structure

The remainder of this report is structured as follows:

- Sections 2–4 position the architecture and define model tiers and signal structures.
- Sections 5–7 define signal lifecycle behavior, dataset formation, and routing policies.
- Sections 8–10 describe runtime interaction, evaluation, and dataset preparation.
- Section 11 defines dataset packaging and pre-freeze preparation.
- Section 12 defines the dataset freeze boundary and inspection-ready artifact structure.
- Sections 13–14 describe alignment with Working-PoD lifecycle and governance frameworks.
- Sections 15–17 provide implementation guidance, limitations, and conclusions.

This structure reflects the progression from signal generation to governed dataset lifecycle execution while preserving separation between reasoning, artifact state, and governance authority.

---

2. Positioning of TR-008

2.1 TR-008 as an Execution-Oriented Extension

TR-008 is defined as an execution-oriented extension to the governance and reasoning constructs established in the Access-PoD research series.

This report does not introduce:

- a governance framework,
- a certification model,
- or a compliance mechanism.

Instead, it defines how governed dataset artifacts may be formed, structured, and prepared for inspection within AI-assisted training environments.

TR-008 operates at the level of:

- signal capture,
- dataset structuring,
- lifecycle routing,
- and packaging and freeze preparation.

It provides a disciplined pathway between:

- AI-assisted reasoning outputs, and
- governed dataset artifacts suitable for inspection and controlled lifecycle progression.

This positioning ensures that TR-008 remains:

- non-normative,
- model-agnostic,
- and subordinate to governance authority defined in earlier publications.

2.2 Separation of Reasoning, Artifact State, and Dataset Lifecycle

TR-008 explicitly maintains separation between three distinct dimensions:

1. Reasoning Posture (Access-Mode — TR-006)
Controls how AI systems reason about governance materials.
Defines permitted assumptions, certainty, and conclusions under:

- ID_1 (learning),
- ID_2 (operational),
- ID_3 (inspection).

2. Artifact State (Working-PoD Lifecycle — TR-006-E.2)
Describes the condition of artifact sets as:

- draft,
- populated,
- or frozen.

Artifact state does not imply correctness, authority, or readiness.

3. Dataset Lifecycle (TR-007)
Defines how datasets evolve across:

- STATE_3 (runtime / synthetic),
- STATE_2 (validation),
- STATE_1 (durable training).

Lifecycle state determines permitted usage and authorization scope.

TR-008 operates across these dimensions without collapsing them.

It:

- consumes reasoning outputs produced under ID_2,
- produces artifact sets that may be prepared under ID_3,
- and routes datasets within the lifecycle defined in TR-007.

This separation ensures that:

- Reasoning behavior does not imply artifact readiness.
- Artifact completeness does not imply dataset authority.
- Dataset lifecycle progression does not imply correctness or approval.

2.3 Signal Exchange as a Non-Authoritative Interaction Layer

TR-008 introduces signal exchange as the primary interaction mechanism between model tiers.

Signals are treated as:

- structured reasoning outputs,
- provisional artifacts,
- and non-authoritative inputs to dataset formation.

Signals shall not be interpreted as:

- decisions,
- approvals,
- or validated conclusions.

This constraint preserves the principle that:

signals represent reasoning activity, not governance authority.

Within this model:

- higher-capacity models may generate **upstream signal artifacts**,
- lower-capacity models may interpret and structure those signals,
- and all signals remain subject to:
    - lifecycle containment,
    - authorization filtering,
    - and validation processes.

Signal exchange therefore enables:

- interaction across model tiers,
- structured dataset formation,
- and controlled evolution of dataset artifacts,

without transferring authority between systems.

2.4 Execution Layer vs Governance Authority

TR-008 explicitly separates:

- **execution behavior**, and
- **governance authority**.

Execution systems:

- capture signals,
- structure datasets,
- and implement routing and packaging logic.

They do not determine:

- whether a dataset is authorized,
- whether a dataset is suitable for training,
- or whether outputs are correct, compliant, or acceptable.

These determinations remain external to TR-008 and must be governed by:

- organizational policies,
- governance frameworks,
- or regulatory processes.

This separation aligns with TR-007, which establishes that:

- execution environments may consume datasets,
- but do not define governance authority.

TR-008 therefore provides:

- an execution model for dataset formation,
- not a mechanism for governance decision-making.

2.5 Non-Certifying and Non-Authoritative Posture

TR-008 inherits and preserves the non-authoritative posture defined across the Access-PoD series.

Accordingly, this report does not:

- assert compliance,
- certify datasets or models,
- assign trust levels,
- or determine adequacy.

All outputs produced under TR-008:

- are non-certifying,
- remain subject to human judgment,
- and must be interpreted as provisional or inspection-oriented artifacts.

This includes:

- signals,
- candidate datasets,
- dataset packages,
- and frozen artifact sets.

The presence of a structured dataset or a completed freeze process shall not be interpreted as:

- approval,
- readiness for deployment,
- or validation of correctness.

2.6 Transition to Architectural Definition

Having established the positioning of TR-008 within the broader governance model, the next section defines the architectural structure supporting this execution model.

Section 3 introduces:

- the tiered model architecture,
- the separation of runtime and governance layers,
- and the signal-first design principles that underpin dataset formation and routing.

---

3. Architectural Overview

3.1 Overview of the Tiered Signal Architecture

TR-008 defines a tier-aware signal architecture for AI-assisted dataset formation.

The architecture is realized in the current reference model through three complementary execution paths:

- **Path 1 - TR corpus / reference path**
- **Path 2 - Access-Mode ID_2 to ID_3 canonical lifecycle path**
- **Path 3 - external or custom dataset adapter path**

Across these paths, three interacting layers remain constant:

- runtime signal generation layer
- governance and routing layer
- dataset formation and packaging layer

Signals generated within runtime environments are treated as structured artifacts that may be:

- captured,
- filtered,

- validated,
- and transformed into candidate datasets.

The architecture ensures that all transformations occur under:

- containment constraints,
- explicit authorization scope,
- and lifecycle-aware routing.

3.2 Tiered Model Interaction Structure

The architecture supports interaction between multiple model tiers.

These tiers are defined by capability, not authority.

Typical tiers include:

- **enterprise-tier models**
  (e.g., 12B–14B class, primary runtime systems)
- **strategic-tier models**
  (e.g., higher-capacity optional reference systems used for contained signal amplification)

Within this structure:

- models may generate signals that function as upstream signal artifacts,
- other models may interpret and structure those signals as downstream signal processors,

These roles are context-dependent and are assigned at the level of signal interaction, not fixed to a specific model tier.

Where enabled, strategic-tier models may act as sources of upstream signal artifacts due to higher representational capacity, while enterprise-tier models commonly perform signal structuring and dataset formation in the released reference model.

However, no model tier inherently possesses authority, and all signals remain subject to governance controls, lifecycle containment, and authorization constraints.

Direct authority transfer between tiers is not permitted.

All cross-tier interaction must occur through:

- signal capture,
- structured records,

- and governance-controlled routing.

3.3 Signal-First System Design

TR-008 adopts a signal-first design principle.

In this model:

- signals are the primary system artifact,
- datasets are derived artifacts,
- and models do not exchange authority directly.

A signal represents:

- a structured reasoning output,
- contextual interpretation,
- or scenario-based observation.

Signals are:

- non-authoritative and subject to governance constraints,
- context-bound,
- and lifecycle-governed.

This approach ensures that:

- reasoning outputs are captured as explicit artifacts,
- dataset formation remains traceable,
- and downstream processes operate on structured inputs rather than implicit model behavior.

3.4 Separation of Runtime, Governance, and Dataset Layers

The architecture enforces strict separation between:

Runtime Layer

- generates signals through AI-assisted reasoning
- operates under ID_2 (operational reasoning posture)
- produces exploratory, uncertainty-aware outputs

Governance Layer

- applies routing, validation, and authorization controls

- enforces lifecycle constraints and containment principles
- maintains dataset passports, lineage records, and authorization scope

Dataset Layer

- structures signals into machine-readable datasets
- produces candidate datasets and package artifacts
- prepares datasets for lifecycle progression and inspection

This separation ensures that:

- runtime behavior does not determine governance outcomes,
- governance rules are not embedded in model execution,
- and dataset artifacts remain traceable and controlled.

3.5 Signal-to-Dataset Transformation Pipeline

The architecture defines a structured transformation pipeline:

```
runtime signal capture
→ signal structuring
→ candidate dataset formation
→ admission and validation
→ dataset packaging (ID_2 snapshot)
→ pre-freeze preparation (normalization + handoff manifest)
→ prepared freeze input (freeze-ready artifact record)
→ ID_3 freeze boundary (frozen package output)
```

Each stage introduces additional structure and governance control.

Key characteristics of this pipeline:

- signals are progressively structured into dataset records
- candidate datasets are filtered and validated before admission
- dataset packages preserve:
    - dataset content,
    - governance artifacts,
    - and traceability references
- pre-freeze preparation normalizes governance metadata
- freeze produces an inspection-ready artifact snapshot

The pipeline ensures that dataset evolution is:

- explicit,
- traceable,
- and governed at each stage.

3.6 Dataset Packaging and Freeze Integration

The architecture integrates dataset packaging and freeze as explicit stages.

Pre-freeze produces a prepared freeze manifest and a prepared freeze input record, which together define the exact artifact set consumed by the freeze boundary.

Dataset packaging produces:

- a structured dataset package (ID_2 snapshot)
- including:
    - dataset records,
    - manifest,
    - passport,
    - routing,
    - authorization,
    - and lineage artifacts

Freeze transforms this package into:

- a stable, inspection-ready snapshot under ID_3 posture

The freeze output includes:

- a preserved dataset package snapshot
- inspection-oriented canonical artifacts
- freeze evidence records
- and a freeze manifest binding all components

This structure ensures that:

- dataset content remains unchanged during freeze,
- governance artifacts remain explicit,
- and inspection is supported without altering dataset semantics.

3.7 Execution-Agnostic Design

TR-008 is designed to be execution-agnostic.

The architecture does not depend on:

- specific model implementations,
- training frameworks,
- or deployment environments.

Execution environments may:

- capture signals,
- structure datasets,
- and implement routing logic,

but do not alter:

- governance authority,
- dataset lifecycle semantics,
- or authorization scope.

This allows the architecture to operate across:

- local development environments,
- enterprise systems,
- and future AI training platforms.

3.8 Architectural Constraints and Invariants

The architecture enforces the following invariants:

- **signals are not decisions**
- **dataset formation is non-authoritative**
- **freeze does not imply approval or compliance.**
- **lifecycle state determines permitted usage**
- **authority remains external to execution systems**

These invariants prevent:

- implicit trust escalation,
- misuse of dataset artifacts,
- and conflation of reasoning outputs with governance decisions.

3.9 Transition to Tier Definitions

Having defined the architectural structure, the next section formalizes the model tier definitions and their roles within the signal exchange system.

Section 4 specifies:

- enterprise-tier and strategic-tier models,
- interaction-role signal roles,
- and capability constraints across tiers.

4. Model Tier Definitions

4.1 Purpose of Tier Definition

This section defines the model tier structure used within TR-008.

Model tiers are introduced to:

- differentiate model capabilities,
- support structured signal interaction across systems of varying capacity,
- and enable controlled dataset formation under governance constraints.

Model tiers are not indicators of:

- authority,
- correctness,
- trust,
- or governance status.

They describe execution characteristics only.

4.2 Enterprise-Tier Models

Enterprise-tier models represent the primary runtime systems within the TR-008 architecture.

Typical characteristics include:

- parameter class in the range of 12B–14B,
- local or enterprise-managed deployment,
- direct interaction with runtime environments,
- and responsibility for signal capture and dataset structuring.

Enterprise-tier models are responsible for:

- generating structured signals from runtime interactions,
- interpreting signals produced by other systems,
- filtering and organizing signals into candidate dataset artifacts,
- and operating under lifecycle containment constraints.

Enterprise-tier models operate under:

- ID_2 (operational reasoning posture),
- and produce signals that are exploratory, conditional, and non-authoritative.

4.3 Strategic-Tier Models

Strategic-tier models represent higher-capacity internal or future-facing reference systems that may be invoked under controlled conditions when enabled.

Typical characteristics include:

- larger parameter classes (e.g., 24B–30B or higher),
- optional, indirect, or API-mediated invocation,
- and use for internal signal amplification, synthesis, or extended reasoning under containment.

When enabled, strategic-tier models may:

- generate structured reasoning signals,
- produce alternative interpretations,
- and contribute to internal scenario exploration under containment.

Any resulting signals are treated as:

- **upstream signal artifacts,**
- non-authoritative and subject to governance constraints,
- and subject to lifecycle routing and validation by enterprise-tier controls.

Within the present release, strategic-tier models do not:

- define governance outcomes,
- operate as required release components or override enterprise-tier systems,
- or transfer authority to downstream processes.

4.4 Interaction-Role Signal Model

TR-008 defines an interaction-role signal model to describe how systems participate in signal generation and dataset formation.

Within this model:

- signal originators generate structured reasoning outputs,
- signal interpreters transform those outputs into governed dataset artifacts.

These roles are:

- contextual to each interaction,

- dynamically assigned
- and independent of model tier, capability, or system identity.

These roles do not imply hierarchy, authority, correctness, or trust.

In practice:

- when enabled, strategic-tier services may commonly produce upstream signal artifacts,
- while enterprise-tier systems commonly act as downstream signal processors in the present release,
- but this relationship is not fixed.

All signals, regardless of origin, remain:

- provisional,
- non-authoritative and subject to governance constraints,
- and subject to governance controls.

4.5 Capability vs Authority Separation

A core principle of TR-008 is the separation of capability and authority.

Model capability refers to:

- computational capacity,
- reasoning complexity,
- and representational power.

Authority refers to:

- governance decision-making,
- authorization of dataset use,
- and determination of correctness or adequacy.

Within this architecture:

- Capability does not imply authority.
- and higher-capacity models do not possess elevated governance status.

All outputs produced by any model tier are:

- treated as signals,
- subject to validation,
- and governed through lifecycle and authorization mechanisms.

4.5.1 Interaction Role Constraint

The interaction-role model defined in this report must be interpreted strictly as a signal-processing abstraction.

It shall not be interpreted as:

- a hierarchy of models,
- a trust gradient between systems,
- a correctness ranking,
- or a delegation of authority.

All signal interactions remain:

- non-authoritative and subject to governance constraints,
- subject to lifecycle containment,
- and governed externally to the execution system.

Capability differences between model tiers do not alter governance authority or trust status.

4.6 Parameter Class vs Deployment Profile

Model tiers are defined independently of deployment environment.

A model's parameter class:

- describes its capacity,
- but does not determine its operational role.

Deployment profiles may include:

- local execution,
- enterprise infrastructure,
- or remote model access.

TR-008 does not require:

- a specific deployment architecture,
- or a specific model provider.

The same model class may be deployed differently across environments while maintaining the same tier classification.

4.7 Controlled Cross-Tier Interaction

Interaction between model tiers is strictly controlled.

All cross-tier interaction must occur through:

- structured signal artifacts,
- dataset records,
- and governance-controlled routing.

Direct interaction patterns such as:

- shared memory,
- implicit state transfer,
- or direct model-to-model control

are not permitted within this architecture.

This ensures the following:

- all interactions are traceable,
- signals remain inspectable,
- and governance controls can be applied consistently.

Cross-tier interaction must be understood as structured signal exchange only. No system inherits authority, trust, or correctness from another system through interaction.

4.8 Prevention of Cross-Tier Authority Transfer

TR-008 explicitly prohibits authority transfer between model tiers.

This includes:

- interpreting higher-capacity model outputs as authoritative conclusions,
- allowing one model tier to override another,
- or treating signals as decisions.

All signals must remain:

- subject to validation,
- context-bound,
- and governed by lifecycle constraints.

This constraint prevents:

- implicit trust escalation,

- over-reliance on model outputs,
- and misuse of signal artifacts.

4.9 Transition to Signal Structure

Having defined the model tier structure and interaction roles, the next section defines the signal record and schema contract.

Section 5 specifies:

- the structure of signal records,
- governance flags and authorization fields,
- and the requirements for traceability and lifecycle integration.

---

5. Signal Record and Schema Contract

5.1 Purpose of the Signal Record

TR-008 defines the signal record as the primary execution artifact within the tiered model signal architecture.

A signal record captures structured reasoning output generated during runtime or controlled model interaction and preserves the minimum information required for:

- traceability,
- lifecycle routing,
- authorization-aware dataset formation,
- and later inspection.

The signal record ensures that model outputs are not treated as implicit behavior or transient responses only. Instead, they are preserved as explicit artifacts that may be:

- reviewed,
- filtered,
- transformed,
- and routed under governance constraints.

Signal records are informative execution artifacts.
They do not imply correctness, adequacy, approval, or authority.

5.2 Signal as the Primary System Artifact

Within TR-008, the signal is the primary system artifact from which downstream dataset structures are derived.

This means that:

- model interaction is recorded through signals,
- routing decisions operate on signals,
- candidate datasets are formed from signals,
- and lifecycle progression begins with signal capture.

This design prevents uncontrolled movement from model output to training artifact.

A signal may contain:

- structured summaries,
- contextual observations,
- scenario reasoning outputs,
- governance-relevant annotations,
- or model invocation metadata.

Signals remain provisional until they are:

- validated,
- filtered,
- and transformed into candidate dataset artifacts under lifecycle control.

5.3 Core Signal Record Structure

A signal record should contain a stable core structure sufficient to support execution, routing, and later audit.

Typical fields include:

- signal identifier
- timestamp
- session identifier
- interaction type
- source initiator/amplifier identifiers
- input prompt or context reference
- structured signal content
- metrics or maturity indicators
- governance flags
- lifecycle declaration
- optional routing, passport, and authorization references

This structure aligns with the execution posture already established in the project build, where runtime records preserve signal identity, governance flags, and lifecycle state as explicit fields rather than implied context.

The purpose of this minimum structure is to ensure that:

- signal origin is visible,
- signal meaning is bounded,
- and downstream tooling can apply deterministic routing and validation.

5.4 Governance Flags and Constraint Fields

Each signal record should preserve governance flags that constrain interpretation and downstream use.

Typical governance flags include:

- provisional status
- context-bound status
- validation requirement
- promotion blocking
- and related containment indicators

These flags exist to ensure that the signal remains:

- non-authoritative and subject to governance constraints,
- non-certifying,
- and unsuitable for automatic progression into durable training datasets.

Where signals originate from exploratory or operational reasoning, governance flags should preserve the same non-authoritative discipline emphasized in Access-Mode and the dataset containment principle in TR-007.

Signal records must not be shaped in a way that allows governance status to be inferred merely from structural completeness.

5.5 Model Invocation and Interaction Metadata

Signal records may include model invocation metadata sufficient to describe how a signal was produced.

Examples include:

- initiating model class

- amplification or secondary model involvement
- deployment profile
- interaction type
- runtime environment reference
- version or registry hints

This metadata is informative and execution-supporting.

It does not:

- define governance authority,
- alter authorization scope,
- or change lifecycle state.

Its purpose is to support reproducibility, later review, and controlled execution analysis without converting execution details into governance decisions.

5.6 Lifecycle Declaration Within the Signal Record

Each signal record should preserve an explicit lifecycle declaration.

In the TR-008 execution model, newly captured signals are ordinarily treated as provisional and routed into a contained lifecycle posture aligned to **STATE_3** semantics, consistent with TR-007's default treatment of newly onboarded datasets as provisional runtime or synthetic resources.

Lifecycle declaration within the signal record should therefore make visible:

- current lifecycle state
- eligibility for review or validation
- promotion blocking conditions
- and any routing-relevant maturity or convergence indicators

This ensures that signals can be interpreted by lifecycle tooling without requiring implicit assumptions.

5.7 Signal Maturity as a Diagnostic Classification

TR-008 permits the use of maturity indicators within signal records as diagnostic metadata.

Signal maturity is used to support:

- routing prioritization,
- validation sequencing,

- and convergence assessment during transition into candidate datasets.

Signal maturity does not imply:

- correctness,
- sufficiency,
- trust,
- or authorization.

It remains a non-authoritative classification used for operational handling only.

This distinction is important because diagnostic maturity can support dataset formation while remaining separate from governance decision-making.

5.8 Lineage and Traceability Requirements

Signal records must support traceability across transformation stages.

At minimum, a signal should preserve enough information to allow later reconstruction of:

- where the signal originated,
- which session or interaction produced it,
- which model role generated it,
- what governance flags applied at capture,
- and how it was later transformed into dataset artifacts.

Where signals contribute to candidate dataset formation, lineage continuity should be preserved through references into:

- candidate manifests,
- dataset passports,
- routing records,
- and lineage records.

This aligns with TR-007's treatment of lineage as informative traceability documentation rather than as a signal of approval or trust.

5.9 Non-Authoritative Signal Constraints

TR-008 requires that signal records preserve explicit non-authoritative constraints.

The origin of a signal does not confer authority. Signals produced by higher-capacity systems must be treated identically to all other signals within the lifecycle and routing model.

A signal record shall not be interpreted as:

- a governance decision,
- a dataset approval,
- a promotion event,
- or a compliance determination.

To preserve this boundary, signal records should be structured and labeled so that:

- signals remain distinguishable from dataset passports,
- execution metadata remains distinguishable from governance metadata,
- and provisional records remain distinguishable from frozen artifacts.

This protects the principle that signals are execution artifacts only.

5.10 Transition to Signal Lifecycle and Routing

Having defined the signal record and schema contract, the next section describes how captured signals progress through lifecycle states and how those states are mapped to governed dataset behavior.

Section 6 defines:

- signal lifecycle mapping,
- containment and promotion constraints,
- and the relationship between signal capture and dataset state progression.

---

6. Signal Lifecycle and Dataset State Mapping

6.1 Purpose of Lifecycle Mapping

This section defines how captured signal records are mapped into governed dataset lifecycle states.

The purpose of lifecycle mapping is to:

- prevent uncontrolled transformation of signals into training datasets,
- preserve dataset containment and authorization boundaries,
- ensure traceable progression from signal to dataset artifact,
- and enforce governance constraints prior to any dataset promotion.

Signal lifecycle mapping provides the operational bridge between:

- signal capture (Section 5), and
- dataset formation and routing under the lifecycle defined in TR-007.

6.2 Signal-to-Dataset Transformation Boundary

Signals represent the earliest structured form of AI-assisted reasoning outputs.

At the point of capture:

- signals are treated as **provisional artifacts**,
- signals are not considered datasets,
- and signals must not be used directly for training or durable storage.

Transformation into dataset artifacts occurs only through:

- structured conversion,
- governance validation,
- and lifecycle-aware routing.

This boundary ensures that:

- model outputs do not bypass governance controls,
- dataset formation is explicit and traceable,
- and dataset artifacts retain linkage to their originating signals.

6.3 Default Lifecycle Assignment (STATE_3)

All captured signals are assigned to a provisional lifecycle state.

Within TR-008, this corresponds to:

STATE_3 — runtime / synthetic dataset layer

Signals entering STATE_3 are:

- context-bound,
- non-authoritative and subject to governance constraints,
- and intended for runtime reasoning or synthetic dataset preparation.

This aligns with dataset lifecycle behavior where newly onboarded artifacts remain under containment and are not eligible for durable training use.

Example characteristics of STATE_3 artifacts include:

- signals derived from ID_2 operational reasoning
- datasets generated from ID_3 frozen artifacts but not yet validated
- exploratory or scenario-based reasoning outputs

STATE_3 is therefore the **default entry point** for all signals.

6.4 Containment and Non-Authoritative Constraints

All signals and derived dataset artifacts shall remain under containment until explicitly promoted.

Containment enforces the following:

- signals_not_decisions = true
- non_certifying posture
- non_authoritative interpretation
- human_decision_required

These constraints are preserved through governance flags and dataset passports.

For example, authorization and filtering controls may prevent signals from being used for fine-tuning or external training while still permitting runtime reasoning and synthetic dataset generation.

This ensures the following:

- Dataset presence does not imply authority.
- Reasoning signals remain contextual.
- Lifecycle progression is governed rather than automatic.

6.5 Lifecycle States and Dataset Roles

TR-008 adopts the three-state lifecycle model defined in TR-007:

STATE_3 — Runtime and Synthetic Dataset Layer

- provisional dataset artifacts
- supports runtime reasoning and synthetic dataset generation
- not authorized for durable training
- remains under containment

STATE_2 — Validation and Convergence Layer

- signals undergo validation and convergence review

- signal alignment is evaluated
- authorization filtering is applied
- dataset readiness is assessed

STATE_1 — Promoted Training Preparation and Durable Training Dataset Layer

- dataset authorized for persistent training use and controlled training preparation export
- requires:
    - explicit governance approval
    - authorization update
    - dataset version issuance

STATE_1 is not an entry state; it is reached only after explicit validation, promotion, and governed handoff.

6.6 Routing and Lifecycle Transition Logic

Lifecycle transitions are governed through routing decisions.

Routing is:

- deterministic,
- governance-first,
- and fail-closed.

Routing decisions evaluate:

- signal structure and schema validity
- governance flags
- authorization scope
- validation status
- maturity indicators

Example routing behavior:

- default → HOLD_IN_STATE_3
- validated candidate → SEND_TO_STATE_2_REVIEW
- invalid signal → REJECT_SIGNAL

Routing records preserve:

- current lifecycle state
- routing decision
- authorization checks
- containment checks

- promotion controls

For example, routing logic may explicitly block promotion due to missing fine-tuning authorization or required human review.

6.7 Authorization and Filtering Constraints

Authorization operates independently of validation and maturity.

Key principles:

- Validation does not imply authorization.
- Review does not imply training permission.
- authorization must be explicitly recorded

Authorization filtering may:

- remove ineligible dataset elements
- restrict dataset usage scope
- prevent promotion to durable training datasets

For example:

- fine-tuning may be disabled
- external training prohibited
- human review required

This ensures that dataset artifacts remain aligned with governance constraints before lifecycle progression.

6.8 Signal Convergence (STATE_2)

STATE_2 functions as the convergence layer between:

upstream signal artifacts (structured reasoning outputs)
and downstream dataset structuring

In this stage:

- signals are evaluated for consistency
- reasoning pathways are compared
- contradictions and uncertainty are surfaced

Convergence results may:

- support further dataset structuring
- trigger additional validation
- or block progression

Convergence does not produce authority.
It supports validation only.

6.9 Dataset Promotion Constraints

Promotion from STATE_3 to STATE_1 is strictly controlled.

Promotion requires:

- successful validation in STATE_2
- authorization update
- dataset filtering
- issuance of a new dataset passport
- explicit promotion event

Promotion must:

- create a new dataset version
- preserve lineage references
- record governance decisions

Direct promotion from STATE_3 to STATE_1 is not permitted.

This ensures the following:

- provisional signals do not influence training without review
- dataset evolution remains traceable
- governance oversight is preserved, and STATE_1 training preparation consumes only promoted datasets.

6.10 Lifecycle Traceability and Lineage

All lifecycle transitions must be traceable.

Traceability is maintained through:

- dataset passports
- routing records
- authorization reports
- lineage records

Lineage records preserve:

- source artifacts (e.g., ID_3 frozen canonical records)
- transformation history
- lifecycle transitions
- policy references

For example, lineage may record:

- ingestion from ID_3 freeze artifacts
- conversion into structured datasets
- routing into STATE_3
- subsequent validation and transformation events

Lineage is informative and does not imply approval or correctness.

6.11 Lifecycle Invariants

The following invariants apply across all lifecycle states:

- signals are not decisions
- dataset artifacts are non-authoritative
- Lifecycle progression does not imply correctness.
- authorization governs usage, not structure
- promotion requires explicit governance action

These invariants ensure that:

- dataset lifecycle remains governed
- reasoning outputs do not become implicit authority
- and dataset evolution remains controlled and auditable

6.12 Transition to Routing and Control Policies

Having defined lifecycle mapping and dataset state behavior, the next section specifies the routing and control policies that enforce these transitions.

Section 7 defines:

- governance-first routing rules
- authorization and validation gates
- deterministic routing behavior
- and escalation and control mechanisms

7. Routing and Control Policies

7.1 Purpose of Routing and Control

This section defines the routing and control model governing how signal artifacts move through the dataset lifecycle.

The purpose of routing is to:

- enforce lifecycle progression rules,
- apply governance constraints prior to any state transition,
- ensure deterministic and auditable signal handling,
- and prevent unauthorized dataset promotion.

Routing operates as the control layer between:

- signal capture and structuring (Sections 3–5), and
- lifecycle progression and dataset formation (Section 6).

Routing decisions are:

- explicit,
- deterministic,
- and governed by fixed evaluation order.

7.2 Governance-First Routing Principle

Routing in TR-008 follows a strict governance-first principle.

Routing decisions must not be influenced by the perceived capability, origin, or interaction role of a signal-producing system.

This means:

- governance constraints are evaluated before signal quality or maturity,
- authorization is evaluated before validation eligibility,
- and lifecycle state transitions are blocked unless all conditions are satisfied.

Routing must not:

- infer readiness from signal structure,
- infer authority from model capability,
- or infer promotion eligibility from maturity classification.

The default routing outcome is:

```
HOLD_IN_STATE_3
```

This conservative default ensures that:

- signals remain contained,
- uncertainty is preserved,
- and governance conditions are not bypassed.

7.3 Deterministic Routing Model

Routing decisions must be deterministic.

For any given signal record:

- the same inputs must produce the same routing outcome,
- evaluation order must not change,
- and routing decisions must be reproducible.

Routing inputs include:

- signal record (schema-compliant)
- governance flags
- authorization block
- validation indicators
- lifecycle fields
- diagnostic metrics and maturity classification

Routing outputs include:

- lifecycle decision (e.g., HOLD, SEND_TO_STATE_2, REJECT)
- updated lifecycle fields
- routing reason codes
- routing log record

Routing must remain:

- model-agnostic
- execution-independent
- and fully auditable

7.4 Fixed Gate Evaluation Order

Routing must follow a fixed, ordered evaluation sequence.

No gate may be skipped or reordered.

Gate A — Structural Admission

Checks:

- schema validity
- required fields present
- lifecycle initialized (STATE_3)
- model metadata present

Failure outcome:

```
REJECT_SIGNAL
```

Gate B — Governance Constraint Check

Checks:

- governance flags present
- signal remains provisional and non-authoritative
- no blocking governance condition

Failure outcome:

```
HOLD_IN_STATE_3 or REJECT_SIGNAL
```

Gate C — Authorization Check

Checks:

- authorization permits candidate progression
- human review requirement recorded
- no authorization conflict

Failure outcome:

```
HOLD_IN_STATE_3
```

Gate D — Validation Trigger Check

Checks:

- validation explicitly requested

- candidate_for_validation = true

Failure outcome:

```
HOLD_IN_STATE_3
```

Gate E — Signal Quality and Maturity Check

Checks:

- maturity threshold for review
- ambiguity within acceptable bounds
- stability sufficient for evaluation

Failure outcome:

```
HOLD_IN_STATE_3
```

Gate F — Transition Encoding

If all gates pass:

- eligible_for_state_2 = true
- routing_decision = SEND_TO_STATE_2_REVIEW

This gate sequence ensures that:

- governance overrides all other signals,
- authorization precedes validation,
- and maturity remains advisory only.

7.5 Routing Outcomes

Routing decisions are constrained to a finite set of outcomes:

- **HOLD_IN_STATE_3**
- **SEND_TO_STATE_2_REVIEW**
- **RETURN_TO_STATE_3**
- **REJECT_SIGNAL**

Rules:

- HOLD_IN_STATE_3 is the default
- SEND_TO_STATE_2 requires full gate satisfaction
- RETURN_TO_STATE_3 applies when validation fails or context changes
- REJECT_SIGNAL applies only for structural or governance failure

All routing outcomes must be:

- explicitly recorded
- accompanied by reason codes
- traceable through routing logs

7.6 Authorization and Validation Separation

Routing enforces strict separation between:

- authorization
- validation
- and signal maturity

Key constraints:

- Validation does not imply authorization.
- Maturity does not imply readiness.
- Review does not imply permission.

For example:

- signals may be eligible for validation
- but still blocked from training due to authorization restrictions
- and require human review before promotion

This separation ensures that:

- dataset formation remains controlled,
- lifecycle progression is governed,
- and training eligibility is not inferred implicitly.

7.7 Governance Flag Enforcement

Governance flags act as hard constraints on routing behavior.

Typical flags include:

- provisional

- context_bound
- requires_validation
- promotion_blocked

Rules:

- governance flags override maturity and metrics
- promotion_blocked must default to true
- routing must check flags before evaluating signal quality

If governance conditions are not satisfied:

- routing must halt progression
- or reject the signal

This ensures the following:

- Signals remain non-authoritative.
- dataset formation remains controlled,
- and lifecycle constraints are preserved.

7.8 Cross-Tier Routing Constraints

Signals originating from cross-tier interaction must follow the same routing rules.

Additional constraints include:

- strategic-tier signals must be interpreted by enterprise-tier before routing
- all cross-tier outputs remain provisional
- no signal inherits authority from model size

Routing must enforce:

- initiator and amplifier traceability
- structured interpretation before artifact creation
- lifecycle containment identical to single-tier signals

This prevents:

- implicit authority transfer
- and uncontrolled influence of higher-capacity models

7.9 Escalation Policy (Enterprise → Strategic)

TR-008 permits controlled escalation from enterprise-tier execution to optional strategic-tier reference services when such services are enabled.

Escalation may occur when:

- ambiguity is high
- signal coherence is low
- alternative reasoning paths are required
- or explicit escalation is requested through controlled configuration

Rules:

- enterprise-tier initiates escalation
- strategic-tier produces candidate signals only
- outputs must return to enterprise-tier for structuring
- all outputs remain in STATE_3 unless separately validated and promoted under lifecycle rules

Within the present release, escalation is:

- explicit,
- reversible,
- non-authoritative and subject to governance constraints,

and optional to successful execution of the released paths.

- This ensures that strategic-tier services act only as internal signal amplifiers or future API reference points,
- not decision-makers or required release components.

7.10 Routing Logs and Traceability

All routing decisions must be recorded.

A routing log must include:

- signal_id
- source_state
- target_state
- routing decision
- reason code
- timestamp

Optional fields may include:

- model parameter class

- interaction type
- escalation usage

Routing logs must be:

- append-only
- immutable
- and fully traceable

This supports:

- auditability
- lifecycle reconstruction
- and governance oversight

7.11 Failure Handling and Conservative Defaults

Routing must implement conservative failure handling.

Rules:

- invalid signal → REJECT
- missing fields → REJECT
- authorization missing → HOLD
- validation uncertain → HOLD
- governance conflict → HOLD or REJECT

When uncertainty exists:

```
do not promote
```

This ensures:

- lifecycle integrity
- prevention of silent failure
- and controlled dataset evolution

7.12 Routing Invariants

The routing system enforces the following invariants:

- governance overrides maturity
- authorization overrides validation
- signals remain provisional by default

- no automatic promotion to STATE_1
- model capability does not affect authority
- all transitions are explicit and logged

These invariants ensure that:

- routing remains governance-first,
- dataset progression remains controlled,
- and system behavior remains interpretable.

7.13 Transition to Runtime Integration

Having defined routing and control policies, the next section describes how runtime systems interact with this architecture.

Section 8 defines:

- signal emission from runtime systems
- execution flow and orchestration
- model endpoint abstraction
- and integration with local and remote environments

---

8. Runtime Integration

8.1 Purpose of Runtime Integration

This section defines how runtime systems generate, structure, and emit signals within the TR-008 architecture.

The runtime layer is responsible for:

- executing model interactions,
- capturing outputs as structured signal records,
- attaching execution and model metadata,
- and preparing signals for routing and lifecycle processing.

Runtime integration provides the operational entry point into the governed dataset lifecycle.

It does not determine governance authority, assign dataset authorization, or perform lifecycle promotion.

All runtime outputs remain:

- ID_2 artifacts,
- provisional,
- and subject to governance constraints.

8.2 Signal Emission from Runtime Systems

Runtime systems must emit all outputs as structured signal records.

Signal emission must follow a consistent transformation pipeline:

```
prompt input
→ model resolution
→ enterprise-tier execution
→ (optional) strategic-tier amplification
→ enterprise-tier interpretation and structuring
→ metric scoring and maturity classification
→ JSON schema binding
→ validation
→ artifact storage
```

All outputs must:

- be captured as JSON signal records,
- conform to the schema defined in Section 5,
- include governance flags and lifecycle fields,
- and be written as immutable artifacts.

No runtime output may bypass:

- schema binding,
- validation,
- or artifact storage.

This ensures the following:

- all reasoning outputs are traceable,
- signals are consistently structured,
- and lifecycle routing can be applied deterministically.

8.3 Enterprise-Tier Execution as Control Layer

The enterprise-tier model acts as the runtime control layer.

Its responsibilities include:

- executing primary model inference,

- structuring all outputs into schema-compliant signal records,
- assigning metrics and maturity classification,
- and enforcing governance flags.

The enterprise-tier model is:

- the sole structuring agent,
- the only layer permitted to produce final signal artifacts,
- and the control point for all cross-tier interactions.

All signals must pass through enterprise-tier structuring before being committed.

This ensures the following:

- signal format is consistent,
- governance flags are applied uniformly,
- and strategic-tier outputs cannot bypass control mechanisms.

8.4 Strategic-Tier Invocation (Optional Layer)

Strategic-tier services, where present, may be invoked under controlled conditions as optional internal reference layers.

Invocation is:

- optional,
- explicit,
- and governed by runtime configuration without being required for the released paths.

When enabled, strategic-tier services may:

- generate candidate reasoning signals,
- provide alternative interpretations,
- or expand internal scenario exploration under containment.

However:

- strategic-tier outputs are not final artifacts,
- must return to enterprise-tier for structuring,
- and remain provisional internal candidate material.

Rules:

- no direct output from strategic-tier to artifact layer
- no authority transfer

- no lifecycle progression without enterprise-tier interpretation and lifecycle controls

If strategic-tier is unavailable:

- runtime continues using enterprise-tier only with no loss of core release function

This preserves system stability and governance consistency across the released reference model.

8.5 Pipeline Orchestration and Execution Flow

Runtime execution is orchestrated through a controlled pipeline.

Core orchestration responsibilities include:

- coordinating model execution
- invoking optional cross-tier interactions
- enforcing execution order
- triggering validation and storage

Execution must be:

- sequential and deterministic
- governed by runtime configuration
- and reproducible across runs

The pipeline must ensure:

- no stage is skipped
- no implicit transitions occur
- and all outputs are recorded

Orchestration logic must not:

- embed governance decisions
- alter lifecycle state
- or bypass routing controls

8.6 Model Endpoint Abstraction

TR-008 requires abstraction of model endpoints so released execution paths and optional internal reference services can be resolved without changing governance behavior.

The present release supports:

- local execution for released signal-generation, reference-baseline, and enterprise-target workflows
- optional remote or API-mediated invocation for internal strategic reference services
- interchangeable endpoint mapping under explicit configuration

Model resolution in the release should be based on:

- released branch or parameter class
- runtime role (enterprise / strategic where applicable)
- registry or configuration

Model identity, hosting location, or endpoint type must not:

- influence governance behavior
- alter signal interpretation

or bypass lifecycle controls

- This abstraction ensures:
- portability across environments
- consistent signal structure across released paths and optional internal references

---

8.7 Local vs Remote Execution

The current release operates primarily in local environments for released signal-generation, reference-baseline, and enterprise-target execution.

- Remote or API-based systems may be used for optional internal reference services
- or future strategic access layers
- Execution mode must be:

explicit within signal metadata

- consistent across runs
- and independent of governance logic
- Differences in execution environment must not:

affect lifecycle entry state

- change authorization scope
- introduce authority
- or alter promotion rules

All outputs used in released paths must remain:

- structurally compatible

- governance-consistent
- and lifecycle-compatible

8.8 Runtime Configuration and Control Parameters

Runtime behavior is governed by configuration.

Typical configuration controls in the current release include:

- released reference or enterprise model enablement
- optional strategic reference-service invocation flag
- model registry path
- validation enablement
- signal storage path

Configuration must:

- be explicit and versioned
- not introduce authority
- and not alter governance constraints

Runtime configuration controls execution behavior only.

It must not:

- override lifecycle rules
- modify authorization scope
- or convert optional reference services into required release dependencies

---

8.9 Failure Handling and Fallback Behavior

Runtime systems must implement explicit failure handling.

Failure conditions include:

- model invocation failure
- validation failure
- storage failure
- or configuration errors

Rules:

- no silent failures
- all failures must be logged

- conservative continuation is default

Fallback behavior:

- strategic-tier failure → continue with enterprise-tier
- validation failure → reject artifact
- storage failure → retry or log failure

When uncertainty exists:

```
do not promote
```

This ensures the following:

- runtime remains stable
- signal integrity is preserved
- and governance boundaries are not violated

8.10 Signal Artifact Storage

All signal records must be stored as immutable artifacts.

Storage requirements:

- append-only behavior
- no overwrite
- consistent naming convention
- structured directory organization

Artifacts must include:

- full signal record
- model metadata
- governance flags
- lifecycle fields

Storage ensures:

- traceability
- reproducibility
- and audit readiness

8.11 Runtime Invariants

The runtime layer must preserve the following invariants:

- all outputs are signals, not decisions
- enterprise-tier is the control layer
- strategic-tier is optional and non-authoritative
- no output bypasses schema binding or validation
- Model capability does not imply authority.
- runtime does not determine lifecycle progression

These invariants ensure that:

- execution remains controlled
- Signals remain non-authoritative.
- and governance authority is preserved

8.12 Transition to Dataset Formation

Having defined runtime integration and signal emission, the next section describes how signals are transformed into candidate datasets under lifecycle control.

Section 9 defines:

- candidate formation rules
- dataset structuring
- validation requirements
- and controlled fine-tuning inputs

---

9. Candidate Formation and Fine-Tuning Inputs

9.1 Purpose of Candidate Formation

This section defines how structured signal records are transformed into candidate dataset artifacts suitable for controlled model adaptation.

Candidate formation provides a governed pathway from:

- provisional signal artifacts (STATE_3),
  to
- structured dataset candidates eligible for validation (STATE_2),
  and eventually
- durable training datasets (STATE_1), subject to authorization.

The purpose of candidate formation is to:

- prevent direct training on raw runtime signals,
- enforce governance constraints prior to dataset use,
- preserve traceability between signals and datasets,
- and support controlled dataset evolution.

Candidate formation is:

- explicit,
- traceable,
- and subject to lifecycle and authorization controls.

9.2 Separation of Signals and Training Data

TR-008 enforces a strict boundary between:

- **signals (runtime artifacts)**
- and **training datasets (durable artifacts)**

Signals:

- are provisional
- context-bound
- and subject to governance constraints.

They must not be used directly for:

- fine-tuning
- training dataset creation
- or external model updates

This boundary ensures that:

- reasoning outputs do not become implicit training data
- dataset formation remains governed
- and lifecycle containment is preserved

Only signals that pass through:

- validation,
- authorization filtering,
- and lifecycle progression

may contribute to dataset candidates.

9.3 Candidate Eligibility Criteria

A signal may be considered eligible for candidate formation only when all of the following conditions are satisfied:

- schema validation passed
- governance flags reviewed
- authorization permits candidate use
- validation status indicates readiness
- lifecycle routing permits progression
- lineage references are preserved

Signals that fail any condition must:

- remain in STATE_3,
- be held for further review,
- or be rejected.

Eligibility must be:

- explicit
- and recorded

No implicit eligibility is permitted.

9.4 Source Eligibility Constraints

Eligible sources for candidate formation include:

- validated signal records
- structured summaries
- reviewed reasoning outputs
- authorized transformation artifacts

Ineligible sources include:

- rejected signals
- schema-invalid records
- context-expired artifacts
- unauthorized signals

Signals originating from STATE_3 remain ineligible until:

- explicitly routed to validation (STATE_2)

- and approved through governance controls

This ensures the following:

- dataset inputs are filtered
- and lifecycle integrity is maintained

9.5 Candidate Dataset Types

TR-008 supports multiple candidate dataset types.

Enterprise Structuring Dataset

Purpose:

- improve structured interpretation
- improve schema compliance
- improve governance-aware outputs

Typical mappings:

- prompt → structured_summary
- prompt → governance_flags
- prompt → lifecycle-aware output

Strategic Signal Dataset

Purpose:

- improve exploratory reasoning
- expand hypothesis generation
- support contrastive signal formation

Typical mappings:

- prompt → alternate interpretations
- prompt → multi-path reasoning
- prompt → signal expansion

Dataset types are:

- role-specific

- and must not be mixed without explicit transformation control

---

9.6 Candidate Record Structure

Each candidate dataset item must preserve traceability and governance constraints.

A candidate record must include:

- candidate identifier
- source signal reference
- source lifecycle state
- target model role and parameter class
- transformation type
- normalized input/output content
- authorization status
- review status
- lineage references
- timestamp

Rules:

- target model must be defined by parameter class, not vendor
- authorization must explicitly allow candidate use
- review must be completed or explicitly required
- lineage must preserve full traceability

Candidate records must not:

- imply approval
- or imply readiness for training

9.7 Authorization and Review Requirements

Authorization is mandatory for candidate formation.

Key principles:

- validation ≠ authorization
- review ≠ permission
- authorization must be explicitly recorded

Authorization controls define:

- whether candidate formation is allowed

- whether fine-tuning is permitted
- whether external training is allowed

For example:

- fine-tuning may be disabled
- external training prohibited
- human review required

Authorization filtering may:

- remove ineligible dataset elements
- restrict dataset usage scope
- block progression to training

This ensures the following:

- candidate datasets remain governed
- and lifecycle constraints are enforced

9.8 Transformation Discipline

Transformation from signal to dataset must be:

- explicit
- recorded
- and reversible

Allowed transformations include:

- normalization
- structured extraction
- deduplication
- field alignment
- prompt-response pairing

Disallowed transformations include:

- untracked rewriting
- authority inflation
- removal of lineage

All transformations must be logged and traceable.

9.9 Dataset Candidate Packaging

Candidate datasets must be packaged in a structured, portable format.

Typical structure includes:

- dataset records (e.g., JSONL)
- dataset manifest
- transformation logs
- export metadata

Packaging must:

- separate datasets by model role
- be append-only
- be versioned
- and preserve reproducibility

Dataset packaging does not imply:

- training readiness
- or lifecycle promotion

9.10 Fine-Tuning Readiness Gates

Before a dataset may be used for fine-tuning, additional conditions must be satisfied:

- dataset integrity validated
- authorization updated for training
- lifecycle progression completed
- dataset passport issued or updated
- governance review completed

Fine-tuning readiness is not determined by:

- signal maturity
- dataset completeness
- or model capability

It requires explicit governance approval.

9.11 Training System Boundary

TR-008 enforces a strict boundary between:

- dataset formation
- and model training

Runtime systems:

- produce datasets
- but do not perform training

Training systems:

- consume datasets
- but do not define governance

This ensures the following:

- governance remains independent of execution
- dataset lifecycle is preserved
- and training systems do not bypass governance controls

9.12 Traceability and Lineage Requirements

All candidate datasets must maintain traceability.

Traceability must include:

- linkage to source signals
- transformation history
- dataset versioning
- policy references

Lineage records must preserve:

- artifact origins
- lifecycle transitions
- and governance context

Lineage is:

- informative
- and subject to governance constraints.

It supports auditability and dataset reconstruction.

9.13 Candidate Formation Invariants

The following invariants apply:

- signals are not training data
- candidate datasets are non-authoritative
- authorization governs dataset use
- Validation does not imply permission.
- lifecycle progression must be explicit
- no automatic promotion to STATE_1

These invariants ensure that:

- dataset formation remains governed
- training inputs remain controlled
- and authority is not inferred

9.14 Transition to Evaluation and Control

Having defined candidate formation and dataset preparation, the next section defines how datasets and models are evaluated, monitored for drift, and subject to revocation.

Section 10 defines:

- evaluation checkpoints
- drift detection mechanisms
- revocation triggers
- and rollback processes

---

10. Evaluation, Drift, and Revocation

10.1 Purpose of Evaluation and Control

This section defines the evaluation and control framework governing released model behavior after dataset-driven adaptation, controlled runtime updates, or STATE_1 training preparation.

The purpose of evaluation is to:

- compare released reference and enterprise behavior under consistent prompts and signal contracts,
- detect structural and behavioral drift,
- ensure continued adherence to governance constraints,
- and support controlled rollback or revocation where required.

In the present release, evaluation is centered on the released reference baseline and enterprise target branches, with optional internal strategic comparisons when enabled.

It does not:

- certify model correctness,
- imply compliance or approval,
- or assign authority to model outputs.

All evaluation outputs remain:

- ID_2 artifacts,
- non-authoritative and subject to governance constraints,
- and subject to human interpretation.

10.2 Evaluation Posture and Constraints

Evaluation must operate under the same operational reasoning posture as runtime signal capture.

Key constraints:

- evaluation is signal-based, not decision-based
- results are comparative, not absolute
- interpretation remains external to the system

Evaluation must not:

- introduce authority
- override governance flags
- alter lifecycle state directly

This ensures alignment with the principle that evaluation provides input but does not determine outcomes.

10.3 Baseline Definition and Preservation

All evaluation requires a baseline reference.

In the present release, the primary released baseline is the Q-Pathformer reference branch (reference_sparse_e08).

- The released enterprise target (grouped_sparse_e64) is evaluated against that baseline after dataset-driven adaptation or controlled runtime change.

- Optional strategic profiles may be compared separately as internal reference material, but are not required for the release evaluation loop.
- Baseline artifacts must include:

sample prompts

- generated signal records
- metric distributions
- governance flag consistency
- Rules:

baseline must be captured before tuning or enterprise-target update

- baseline must be preserved and immutable
- baseline must be reproducible across the release environment
- baseline scope must be explicit for every comparison run

This enables consistent comparison across released evaluation runs.

10.4 Post-Update Evaluation Process

After dataset-driven adaptation, controlled runtime profile change, or STATE_1 training preparation:

- the relevant released model is re-evaluated using the same baseline prompt set and signal contract
- new signal records or training-run outputs are generated
- outputs are compared against the preserved baseline

Comparison dimensions include:

- signal metrics (coherence, novelty, stability, ambiguity)
- maturity classification distribution
- structural consistency (schema validity)
- governance flag correctness
- lifecycle field consistency

Evaluation must be:

- deterministic
- measurable
- repeatable across released branches and controlled training outputs

Qualitative-only evaluation is not sufficient.

10.5 Signal Quality Metrics and Interpretation

Evaluation uses signal-level metrics defined in Section 5:

- coherence
- novelty
- stability
- ambiguity

Derived indicators may include:

- metric deltas
- variance shifts
- structural failure rate
- governance consistency rate

Rules:

- no single metric determines quality
- trends are more important than isolated values
- metrics remain diagnostic only

Metric interpretation must not:

- imply correctness
- imply approval
- or override governance constraints

10.6 Cross-Tier Evaluation

TR-008 permits comparison between released reference and enterprise outputs, and, where enabled, optional strategic internal reference outputs.

In the present release, primary comparison centers on the reference baseline (reference_sparse_e08) and the enterprise target (grouped_sparse_e64).

- Cross-tier or cross-branch evaluation may assess:
- divergence in signal structure
- variation in ambiguity
- consistency of structured outputs

breadth of reasoning pathways

- Rules:
- comparison is diagnostic only
- higher-capacity or non-release models do not imply correctness

no automatic adoption of strategic-tier outputs or non-release reference outputs

- Cross-tier evaluation must not:
- affect routing decisions directly
- override governance flags or make optional strategic references a required release dependency

This preserves release discipline while enabling analysis.

10.7 Drift Detection

Drift detection identifies changes in model behavior over time.

Drift types include:

Structural Drift

- schema violations
- missing required fields
- inconsistent formatting

Behavioral Drift

- increased ambiguity
- loss of structured discipline
- unstable outputs

Governance Drift

- missing or incorrect flags
- lifecycle misassignment
- emergence of authority-like language

Drift detection is based on:

- baseline comparison
- threshold evaluation
- anomaly detection

Drift signals must be:

- recorded
- traceable
- and reviewable

Drift does not automatically trigger revocation.

10.8 Drift Thresholds and Interpretation

Thresholds may be defined for:

- metric deviation
- structural failure rate
- governance violations

Examples:

- significant increase in ambiguity
- drop in coherence
- schema failure rate above threshold
- governance flag inconsistencies

Rules:

- thresholds are advisory
- thresholds are configurable
- thresholds do not determine authority

Critical drift conditions must trigger review.

10.9 Revocation Triggers

Revocation is triggered when:

- critical drift is detected
- structural failure exceeds acceptable limits
- governance violations occur
- unexpected behavior patterns emerge

Revocation conditions must be:

- explicit
- recorded
- and reviewable

Revocation is a governance action, not an automated conclusion.

10.10 Revocation and Rollback Actions

When revocation is required:

- the affected model version is disabled
- the system reverts to a previous stable version
- the revocation event is recorded
- the failed model is preserved for analysis

Rules:

- no silent rollback
- revoked models must not be reused
- all actions must be traceable

Rollback must:

- restore prior behavior
- preserve lineage
- and maintain auditability

10.11 Continuous Evaluation Loop

Evaluation operates as a continuous loop:

```
baseline
→ dataset formation
→ model adaptation
→ evaluation
→ comparison
→ decision (retain / rollback / revise)
```

Rules:

- evaluation must follow every model update
- no model progresses without evaluation
- decisions must be explicit
- conservative action is the default

This loop ensures that:

- model evolution remains controlled
- dataset impact is measurable
- and governance constraints are preserved

10.12 Governance Safeguards

The evaluation system must enforce:

- evaluation does not create authority
- Passing evaluation does not imply approval.
- outputs remain ID_2 signals
- lifecycle rules cannot be overridden

Evaluation must not:

- promote datasets
- authorize training
- or alter governance status

This maintains separation between:

- execution
- evaluation
- and governance authority

10.13 Storage and Traceability

Evaluation artifacts must be stored in structured release-aware locations:

- released baselines
- released evaluation runs
- optional internal reference comparisons and revoked models where applicable
- logs

Storage must be:

- append-only
- versioned
- and fully traceable

Evaluation records must include:

- model branch or parameter class
- baseline reference
- metric comparison
- drift flags
- structural failures
- governance violations
- timestamp

release-scope label where applicable

- This supports:
- auditability
- reproducibility and lifecycle traceability

10.14 Evaluation Invariants

The following invariants apply:

- Evaluation does not imply correctness or approval.
- Evaluation does not override governance.
- Model capability does not imply authority.
- All results remain diagnostic.

These invariants ensure:

- disciplined interpretation,
- prevention of authority inflation,
- and alignment with Access-PoD principles.

10.15 Transition to Implementation Guidance

Having defined evaluation, drift control, and revocation, the next section provides non-normative implementation guidance.

Section 11 describes:

- infrastructure options
- tooling considerations
- and execution environments

---

11. Implementation Guidance (Non-Normative)

11.1 Purpose of Implementation Guidance

This section provides illustrative guidance for implementing the execution model described in TR-008.

The guidance is:

- non-normative,
- non-prescriptive,

- and intended to support practical adoption.

It does not:

- define requirements,
- mandate specific tools or architectures,
- or establish compliance criteria.

Implementation approaches may vary depending on:

- organizational context,
- infrastructure constraints,
- and deployment objectives.

11.2 Execution Environments

The TR-008 model is execution-agnostic.

It may be implemented across:

- local development environments
- enterprise infrastructure
- distributed systems
- or managed cloud platforms

Typical deployment patterns include:

- **local execution**
  - rapid iteration and development
  - low-latency testing of signal pipelines
- **enterprise deployment**
  - controlled runtime environments
  - integration with internal governance systems
- **distributed or cloud execution**
  - scalable signal processing
  - remote model invocation

Execution environment choice must not:

- alter governance behavior,
- change lifecycle semantics,
- or introduce authority.

11.3 Model Access and Registry Patterns

Implementations may use a model registry to abstract model selection and access.

Typical registry responsibilities include:

- mapping model roles (enterprise / strategic)
- associating parameter classes with endpoints
- defining local vs remote execution paths
- maintaining version references

Model access may be:

- local (on-device or on-premise)
- remote (API-based)
- hybrid (combination of both)

All model access patterns must:

- remain interchangeable
- preserve signal schema consistency
- and avoid embedding governance logic

Model identity must not influence:

- lifecycle state
- routing behavior
- or authorization scope

11.4 Signal Pipeline Implementation

Signal capture and processing may be implemented as a modular pipeline.

Typical components include:

- input handling
- model execution
- optional cross-tier interaction
- signal structuring and normalization
- metric scoring and maturity classification
- schema validation
- artifact storage

Pipelines may be implemented using:

- scripts
- services
- or orchestrated workflows

Regardless of implementation:

- all outputs must be captured as structured signal records
- validation must be enforced
- and artifacts must be stored immutably

11.5 Dataset Handling and Storage

Dataset formation and candidate packaging may be implemented using:

- structured file formats (e.g., JSONL)
- manifest-based packaging
- versioned dataset directories

Typical dataset structures include:

- candidate datasets
- transformation logs
- dataset manifests
- lifecycle logs

Storage systems may include:

- local file systems
- object storage
- database-backed systems

Dataset storage must ensure:

- append-only behavior
- version control
- and full traceability

11.6 Fine-Tuning Approaches (Non-Prescriptive)

TR-008 does not prescribe specific training methods.

Common approaches may include:

- parameter-efficient fine-tuning (e.g., adapter-based methods)
- full fine-tuning
- hybrid approaches

Training may be executed using:

- local training environments
- distributed training pipelines
- or managed services

All training systems must:

- consume governed datasets
- preserve dataset lineage
- and re-enter evaluation pipelines after updates

Training tooling must not:

- bypass lifecycle controls
- alter governance metadata
- or introduce authority

11.7 Hardware and Infrastructure Considerations

Implementation may vary depending on available hardware.

Considerations include:

- CPU vs GPU environments
- memory constraints
- model size compatibility
- latency requirements

Typical patterns:

- enterprise-tier models may operate on local or enterprise hardware
- strategic-tier models may require higher-capacity infrastructure

Infrastructure decisions must not:

- influence governance behavior
- or affect lifecycle constraints

11.8 Integration with External Tooling

Implementations may integrate with existing tooling ecosystems.

Examples include:

- model hosting platforms

- dataset processing tools
- training frameworks
- orchestration systems

Such tools may support:

- rapid experimentation
- dataset transformation
- and model iteration

However:

- tooling remains external to governance logic
- and must not override lifecycle or authorization controls

Integration must preserve:

- signal schema integrity
- dataset traceability
- and governance invariants

11.9 Logging and Observability

Implementations should include logging and observability mechanisms.

Typical logging may include:

- signal capture events
- routing decisions
- validation outcomes
- dataset transformations
- evaluation results

Logs should be:

- append-only
- timestamped
- and traceable

Observability supports:

- debugging
- audit
- and lifecycle reconstruction

11.10 Failure Handling and Recovery

Systems should implement explicit failure handling.

Common failure cases include:

- model invocation errors
- schema validation failures
- storage issues
- routing inconsistencies

Recommended behavior:

- log all failures
- avoid silent continuation
- apply conservative fallback

Where uncertainty exists:

```
do not promote
```

This ensures:

- system integrity
- and governance consistency

11.11 Implementation Boundaries

This section does not define:

- governance authority
- lifecycle rules
- dataset authorization
- or inspection requirements

These are defined in:

- TR-006 (reasoning posture)
- TR-006-E.2 (artifact lifecycle)
- TR-007 (dataset lifecycle)

Implementation guidance supports execution only.

11.12 Transition to Governance Alignment

Having described implementation considerations, the next section addresses alignment with governance frameworks and standards positioning.

Section 12 defines:

- alignment with Access-PoD artifacts
- dataset passport integration
- inspection and audit considerations
- and standards positioning

---

12. Dataset Freeze Boundary (ID_3 Packaging and Archive Freeze)

12.1 Purpose of the Freeze Boundary

This section defines the dataset freeze boundary as applied within the Q-Pathformer execution environment.

The freeze boundary represents the transition from a governed dataset package assembled under operational reasoning conditions to a stable, inspection-ready artifact prepared for external interpretation.

This boundary is aligned with:

- the inspection posture defined in Access-Mode ID_3,
- the artifact lifecycle clarification described in APOD-TR-006-E.2, and
- the dataset containment and lifecycle routing model defined in APOD-TR-007.

The freeze boundary represents the final stage of the execution pipeline defined in this report, following:

- signal capture,
- dataset formation,
- dataset packaging,
- and pre-freeze preparation.

The purpose of the freeze boundary is to:

- preserve dataset structure and governance context as a fixed snapshot,
- ensure deterministic integrity through cryptographic hashing,
- enable inspection and review without altering dataset content or semantics, and
- prevent implicit authority or approval from being inferred from dataset completeness.

12.2 Freeze as an Artifact State Transition

The freeze operation is an explicit artifact-state transition.

Dataset packages assembled under ID_2 operational reasoning posture remain non-inspectable until explicitly transitioned to ID_3 through the freeze boundary.

A dataset is considered frozen only when:

- it is prepared under ID_3 inspection posture,
- the artifact set is treated as issued for review,
- and further modification is suspended or versioned.

Freezing is a deliberate act and must not occur implicitly.

In accordance with APOD-TR-006-E.2:

- freezing indicates that an artifact set is stable enough for inspection,
- but does not imply correctness, adequacy, compliance, or approval.

The freeze boundary therefore represents a transition in artifact state, not a change in governance authority or dataset trust status.

12.3 Relationship to Dataset Lifecycle States

The freeze boundary operates within the dataset lifecycle model defined in APOD-TR-007.

In this model:

- datasets assembled through signal capture and candidate formation ordinarily begin within STATE_3,
- STATE_3 datasets are provisional, context-bound, and governed by containment until review and promotion occur,
- and datasets must not be treated as durable training assets without explicit promotion.

Freezing a provisional STATE_3 artifact does not promote a dataset to STATE_1.

Separately:

- the release also permits downstream STATE_1 handoff and freeze operations after explicit promotion; these do not make STATE_1 an entry state.
- The freeze operation preserves a stable snapshot appropriate to the lifecycle state being frozen.

This distinction ensures that:

- Dataset existence does not imply authorization for training use.
- Inspection readiness does not imply dataset maturity or approval.

12.4 Freeze Input Requirements

A dataset may be frozen only when a complete set of freeze inputs is present.

These inputs include:

1. Dataset Package (ID_2 Snapshot)

- structured dataset (e.g., JSONL)
- dataset manifest
- governance artifacts (passport, routing, authorization, lineage)

2. Prepared Freeze Manifest

- artifact paths resolved for freeze
- governance normalization applied where required
- dataset counts reconciled

Prepared freeze inputs may include normalized governance artifacts to ensure consistency across:

- dataset passport
- routing records
- authorization reports
- and lineage records

prior to hashing and freeze execution.

3. Canonical Inspection Artifacts

- module-level canonical records
- session-level canonical records
- explicitly marked as `inspection_status = frozen_under_ID_3`

4. Expected Hash Manifest

- pre-declared SHA-256 hashes for canonical artifacts

5. Freeze Note

- human-readable declaration of freeze scope and limitations

6. Prepared Freeze Input Record

- prepared artifact set derived from pre-freeze
- serves as the authoritative input boundary for freeze execution
- ensures deterministic artifact resolution independent of original package location

These inputs must be complete before freeze is executed.

Missing or inconsistent inputs must result in failure under fail-closed conditions.

12.5 Frozen Package Structure

The freeze process produces a structured artifact set composed of distinct layers:

```
frozen_package_root/
├── dataset_package_snapshot/   (ID_2 dataset package snapshot)
├── canonical_artifacts/       (ID_3 inspection artifacts)
├── freeze_evidence/          (freeze note and hash manifests)
└── freeze_manifest.json
```

Each layer serves a distinct purpose:

dataset package snapshot

- contains an exact copy of the dataset package assembled under ID_2
- preserves dataset content and governance artifacts without modification
- represents the original ID_2 dataset package snapshot preserved under the ID_3 freeze boundary

canonical artifacts

- contains inspection-oriented canonical artifacts
- supports reviewer interpretation and traceability

freeze evidence

- contains:
    - expected hash definitions
    - computed hash manifest
    - freeze declaration note

freeze_manifest.json

- binds the freeze event to:
    - source package
    - artifact locations
    - hash records
    - and freeze posture

The frozen package is derived from the prepared freeze input record, not directly from the original package assembly location.

This separation ensures that:

- runtime dataset content,
- inspection artifacts,
- and integrity evidence

remain clearly distinguishable.

12.6 Hash Integrity and Verification

Freeze includes a deterministic hashing process.

Hash computation applies to the fully prepared freeze output, including packaged dataset, canonical artifacts, and evidence records.

The system must:

- compute SHA-256 hashes over the frozen artifact set,
- verify canonical artifact hashes against expected values,
- and produce a complete hash manifest for the frozen snapshot.

Hashing is performed:

- after canonical artifacts are finalized,
- and prior to recording the freeze manifest.

Hash verification ensures:

- that canonical artifacts match declared values,
- and that the frozen snapshot can be independently validated.

Failure of hash verification must result in termination of the freeze process under strict mode.

12.7 Governance Semantics of Freeze

The freeze boundary explicitly preserves the following governance invariants:

- signals_not_decisions
- non_certifying
- non_authoritative

- dataset_containment_principle

These invariants ensure that:

- dataset signals remain contextual and non-authoritative,
- Freeze does not imply validation, approval, or compliance.
- and inspection artifacts do not function as governance decisions.

Freeze is therefore an archival and inspection-preparation process applied to governed dataset artifacts under containment.

12.8 Distinction Between Freeze and Promotion

Freeze must not be confused with dataset promotion.

Within the lifecycle model:

Freeze

- produces an inspection-ready snapshot
- preserves dataset state under containment
- does not change authorization scope

Promotion

- transitions datasets between lifecycle states
- may authorize durable training use
- requires governance review and new dataset passport issuance

Freeze operates independently of promotion.

A frozen dataset may:

- remain in STATE_3,
- be re-opened for modification,
- or be used as evidence in governance processes.

It does not automatically progress to STATE_1.

12.9 Freeze Output and Traceability

The output of the freeze process is:

- a frozen dataset package

- a canonical inspection artifact set
- a hash-based integrity record
- and a freeze manifest linking all components

This output supports:

- inspection and review
- traceability across artifact transformations
- and audit of dataset provenance and lifecycle state

All freeze outputs must remain:

- versioned
- traceable
- and resistant to silent modification

---

13. Relationship to Working-PoD Lifecycle (TR-006-E.2)

13.1 Purpose of Lifecycle Alignment

This section clarifies how the execution model defined in TR-008 aligns with the Working-PoD lifecycle described in APOD-TR-006-E.2.

The purpose of this alignment is to:

- distinguish artifact state from reasoning posture,
- prevent misinterpretation of dataset completeness as authority,
- and ensure that dataset packaging and freeze are interpreted consistently with Working-PoD lifecycle definitions.

TR-008 adopts the lifecycle model as a foundational interpretive framework without modifying its semantics.

13.2 Distinction Between Reasoning Posture and Artifact State

The Working-PoD lifecycle defines a separation between:

- **reasoning posture** (controlled by Access-Mode), and
- **artifact state** (describing the condition of governance artifacts).

TR-008 preserves this distinction.

Within this model:

- ID_1, ID_2, and ID_3 define how reasoning occurs,
- while artifact states define what exists at a given point in time.

TR-008 operates primarily under:

- ID_2 (operational reasoning posture) for runtime signal capture and dataset formation,
- and transitions to ID_3 only at the freeze boundary for inspection preparation.

Artifact state must not be inferred from reasoning posture alone.

13.3 Working-PoD Lifecycle States

The Working-PoD lifecycle defines three primary artifact states:

- **Draft State**
- **Populated (Posture-Bound) State**
- **Frozen (Inspection-Ready) State**

These states describe:

- artifact completeness,
- not governance quality or authority.

A Working-PoD may appear structurally complete under ID_2 without being inspection-ready.

Inspection readiness requires:

- explicit freeze under ID_3,
- stable artifact structure,
- and declared scope and limitations.

13.4 Mapping TR-008 Execution to Lifecycle States

TR-008 execution stages map to Working-PoD lifecycle states as follows:

Draft State

Corresponds to:

- early signal capture
- exploratory runtime interactions
- incomplete or evolving artifact sets

Characteristics:

- signals are provisional
- datasets are not yet structured
- governance artifacts may be incomplete

Populated (Posture-Bound) State

Corresponds to:

- structured signal records
- candidate dataset formation
- dataset packaging under ID_2

Characteristics:

- all required structures are present
- artifacts support operational reasoning
- lifecycle and governance fields are populated

However:

- artifacts remain non-inspectable
- and shall not be interpreted as stable or authoritative

Frozen (Inspection-Ready) State

Corresponds to:

- ID_3 freeze boundary defined in Section 12

Characteristics:

- artifact set is explicitly frozen
- structure is stable and enumerated
- traceability and integrity are preserved

Freeze indicates:

- readiness for inspection,
- not correctness or approval.

13.5 Freeze as the Transition to Inspection Readiness

The Working-PoD lifecycle requires that inspection readiness be explicitly declared.

Within TR-008:

- this declaration is implemented through the dataset freeze boundary.

The freeze boundary:

- transitions artifacts from populated to frozen state,
- suspends modification or requires versioning,
- and prepares artifacts for external interpretation.

This aligns with lifecycle requirements that:

- freezing must be explicit,
- and must not occur implicitly through reuse or publication.

13.6 Prevention of Authority Inflation

A key purpose of the Working-PoD lifecycle is to prevent the misinterpretation of artifact completeness as authority.

TR-008 reinforces this by ensuring that:

- structured datasets do not imply correctness,
- populated artifacts do not imply readiness,
- frozen artifacts do not imply approval or compliance.

Even when artifacts are:

- fully structured,
- validated,
- and frozen,

they remain:

- non-authoritative and subject to governance constraints,
- subject to challenge,
- and dependent on human interpretation.

This prevents:

- learning artifacts from being treated as conclusions,
- operational artifacts from being treated as validated outcomes,

- and inspection artifacts from being treated as endorsements.

13.7 Lifecycle Transitions and Explicit Signaling

Transitions between lifecycle states must be explicit.

Within TR-008:

- Signal capture does not imply progression beyond draft.
- Dataset structuring does not imply inspection readiness.
- freeze must be explicitly declared

Each transition must be accompanied by:

- a clear Access-Mode context,
- a statement of intended use,
- and explicit limitation or scope declarations.

Silent transitions — particularly from populated to frozen — must be avoided.

13.8 Relationship Between Lifecycle and Dataset States

The Working-PoD lifecycle and dataset lifecycle operate in parallel but distinct dimensions.

- Working-PoD lifecycle describes **artifact state**
- Dataset lifecycle describes **dataset maturity and authorization**

Within TR-008:

- a dataset may be in STATE_3 while also being in a frozen artifact state
- freeze does not alter lifecycle state
- lifecycle progression requires separate governance actions

This ensures the following:

- Inspection readiness does not imply dataset promotion.
- Dataset maturity does not imply artifact stability.

13.9 Lifecycle Invariants

The following invariants apply:

- Artifact state does not imply authority.
- Reasoning posture does not imply readiness.
- Freeze does not imply correctness or approval.
- Dataset lifecycle does not imply inspection readiness.
- all interpretations remain subject to human judgment

These invariants preserve:

- clarity of artifact meaning
- separation of concerns
- and governance discipline

13.10 Transition to Governance Alignment

Having established alignment with the Working-PoD lifecycle, the next section addresses broader governance alignment and standards positioning.

Section 14 defines:

- alignment with Access-PoD governance artifacts
- dataset passport integration
- audit and inspection considerations
- and standards positioning

---

14. Compliance and Governance Alignment

14.1 Purpose of Governance Alignment

This section defines how TR-008 aligns with the governance framework established in the Access-PoD research series.

The purpose of this alignment is to:

- ensure execution-layer mechanisms operate within defined governance authority,
- prevent misinterpretation of execution artifacts as governance decisions,
- and explicitly bound authority across framework, architecture, containment, and stress layers.

TR-008 operates as an **execution-layer extension** and does not introduce governance authority.

14.2 Authority Boundaries Across Access-PoD Versions

Governance authority within the Access-PoD series is explicitly bounded across editions.

14.2.1 Version A — Governance Authority (Primary Source of Authority)

APOD-TR-002 Version A

Version A defines:

- governance authority
- assurance semantics
- CSC scoring and revocation logic
- evidence-based trust model

Authority characteristics:

- trust is derived from telemetry, not declaration
- assurance is continuous and revocable
- governance operates as a runtime system

All governance authority originates here

TR-008 does not modify or extend these definitions.

14.2.2 Version B — Architecture and Execution Boundaries

APOD-TR-003 Version B

Version B defines:

- execution boundaries
- API control surfaces
- telemetry generation points
- provenance structures

Key constraint:

governance authority is not embedded in architecture

Authority characteristics:

- architecture **executes**, but does not decide

- APIs are **policy-bound nodes**, not authority endpoints
- telemetry is **evidence input**, not assurance

Architecture is an execution substrate, not a governance authority

---

14.2.3 Version C — Containment and Onboarding Discipline

APOD-TR-004 Version C

Version C defines:

- governed entry into runtime systems
- containment as a prerequisite for trust
- onboarding discipline under uncertainty

Core principle:

containment precedes alignment

Authority characteristics:

- onboarding ≠ approval
- dataset presence ≠ trust
- alignment cannot be assumed at entry

Containment constrains behavior before authority is interpreted

---

14.2.4 Version D — Stress and Ethical Constraint

APOD-TR-005 Version D

Version D defines:

- governance under stress
- ethical decision capability
- revocation readiness

Core principle:

stress-testing constrains trust; it does not elevate it

Authority characteristics:

- stress reveals governance limits

- ethical decisions remain non-delegable
- trust must degrade when evidence fails

Stress-testing constrains and validates governance authority

14.3 TR-008 Position Within Authority Structure

TR-008 operates across all four layers but introduces no authority.

It defines:

signal → dataset → lifecycle → package → freeze

TR-008 is therefore:

- an execution-layer orchestration model
- a dataset formation pathway
- a lifecycle-aware transformation system

It is not:

- a governance authority
- a certification framework
- a compliance mechanism

14.4 Dataset Passport and Lifecycle Alignment

TR-008 integrates with the dataset passport system defined in TR-007.

Dataset passports:

- define dataset identity and provenance
- constrain usage through authorization
- bind lifecycle state and containment

Within TR-008:

- signals are transformed into dataset artifacts
- candidate datasets inherit passport constraints
- lifecycle progression is passport-governed

This ensures:

- dataset usage remains constrained

- lifecycle progression remains governed
- authority is not inferred from dataset structure

14.5 Execution vs Governance Authority Separation

TR-008 enforces strict separation:

Layer	Function	Authority
TR-008	execution (signals, datasets, packaging)	none
Version B	execution substrate	none
Version C	containment discipline	none
Version D	stress constraint	none
Version A	governance + assurance	authoritative

Execution systems:

- generate signals
- structure datasets
- apply routing logic

They do not:

- authorize training
- approve datasets
- assign compliance

14.6 Inspection, Freeze, and Evidence Alignment

Inspection readiness is achieved only through:

- ID_3 freeze boundary
- structured canonical artifacts
- deterministic hash verification

Freeze outputs:

- remain non-authoritative
- are inspection-ready, not approved
- preserve containment

This aligns with:

- Working-PoD lifecycle (TR-006-E.2)
- dataset containment (TR-007)

14.7 Standards Positioning

TR-008 is positioned as:

- non-normative
- execution-oriented
- standards-aligned

It supports:

- runtime governance experimentation
- dataset lifecycle structuring
- inspection-ready artifact preparation

It does not:

- define compliance criteria
- issue certifications
- replace regulatory frameworks

14.8 Explicit Non-Claims

The following must not be inferred:

- regulatory compliance
- dataset approval
- model safety
- deployment readiness
- certification status

The presence of:

- structured datasets
- lifecycle routing
- freeze artifacts

does not imply:

- correctness
- validation
- authority

14.9 Governance Invariants Across Versions

Across TR-002 → TR-005 → TR-008:

- trust is evidence-derived
- authority is external to execution
- signals are not decisions
- datasets are not authority
- freeze is not approval
- evaluation is not validation

These invariants ensure:

- governance discipline
- interpretability
- prevention of authority inflation

---

15. Conclusion

15.1 Summary of Contribution

This technical report defines a structured execution model for transforming AI-assisted reasoning outputs into governed dataset artifacts within a lifecycle-constrained environment.

TR-008 introduces:

- a signal-first execution model,
- a tier-aware interaction structure,
- a governed pathway from signal capture to dataset formation,
- and an explicit freeze boundary for inspection-ready artifact preparation.

These elements collectively provide a disciplined method for:

- capturing reasoning outputs as structured artifacts,
- transforming signals into traceable datasets,
- applying lifecycle-aware routing and authorization controls,
- and producing inspection-ready dataset packages without introducing implicit authority.

The model is:

- execution-oriented,
- model-agnostic,

- and compatible with multiple deployment environments.

15.2 Integration with Access-PoD Governance Framework

TR-008 operates within the governance structure established in the Access-PoD research series.

It aligns with:

- Version A — governance authority and assurance semantics,
- Version B — execution boundaries and telemetry generation,
- Version C — containment discipline and onboarding constraints,
- Version D — stress-testing and ethical evaluation.

Within this structure:

- governance authority remains external to execution systems,
- execution systems produce artifacts but do not assign authority,
- and trust is derived from evidence rather than inferred from structure.

TR-008 does not redefine governance principles.

It provides a structured execution pathway that remains compatible with those principles.

15.3 End-to-End Lifecycle Execution Model

The report establishes a complete lifecycle from runtime interaction to inspection-ready dataset artifacts:

```
signal capture
→ signal structuring
→ lifecycle routing (STATE_3 → STATE_2 → STATE_1 constraints)
→ candidate dataset formation
→ dataset packaging (ID_2)
→ pre-freeze preparation
→ prepared freeze input (prepared handoff)
→ ID_3 freeze boundary (frozen package output)
```

This lifecycle ensures that:

- signals remain provisional until validated,
- dataset formation is explicit and traceable,
- authorization governs dataset use,
- and inspection readiness is achieved only through explicit freeze.

The model prevents:

- direct transformation of signals into training data,
- implicit promotion of datasets,
- and uncontrolled dataset evolution.

15.4 Preservation of Governance Boundaries

A central contribution of TR-008 is the preservation of strict boundaries between:

- reasoning posture,
- artifact state,
- dataset lifecycle,
- and governance authority.

These boundaries ensure that:

- Execution behavior does not imply governance decisions.
- Dataset structure does not imply authorization.
- Inspection readiness does not imply approval or compliance.

The freeze boundary reinforces this separation by:

- transitioning artifacts to inspection-ready state,
- without altering lifecycle state or authorization scope.

15.5 Execution Without Authority

TR-008 demonstrates that:

- AI systems can generate structured, traceable datasets,
- lifecycle-aware routing can be applied deterministically,
- and inspection-ready artifacts can be produced

without embedding governance authority into execution systems.

Execution systems:

- capture signals,
- structure datasets,
- and implement routing logic

but do not:

- approve datasets,
- authorize training,

- or determine compliance.

This separation ensures that:

- governance remains accountable,
- execution remains controlled,
- and authority is not inferred from system capability.

Interaction roles describe signal flow, not authority, correctness, or trust relationships between systems.

15.6 Role of Freeze in Lifecycle Integrity

The introduction of the dataset freeze boundary provides:

- a controlled transition to inspection readiness,
- deterministic integrity through hashing,
- and a structured artifact set supporting audit and review.

Freeze:

- preserves dataset state under containment,
- separates runtime artifacts from inspection artifacts,
- and enables external interpretation without altering dataset content.

Freeze does not:

- promote datasets,
- imply validation,
- or confer authority.

It represents an artifact-state transition aligned with the Working-PoD lifecycle.

15.7 Invariants of the TR-008 Model

The following invariants apply across the model:

- signals are not decisions
- datasets are not authority
- Lifecycle progression does not imply correctness.
- Validation does not imply authorization.
- Freeze does not imply approval.
- Evaluation does not imply certification.

These invariants ensure:

- interpretive clarity,
- governance discipline,
- and prevention of authority inflation.

15.8 Standards and Practical Implications

TR-008 contributes to AI governance practice by:

- enabling structured dataset lifecycle execution,
- supporting evidence-based governance workflows,
- and providing a reference model for signal-driven dataset formation.

The model supports:

- standards exploration,
- governance system design,
- and practical implementation in AI training environments.

It remains:

- non-normative,
- non-certifying,
- and execution-focused.

15.9 Final Statement

TR-008 demonstrates that AI systems can:

- operate within a structured, governed lifecycle,
- produce inspection-ready artifacts,
- and support controlled dataset evolution

without conflating execution with authority.

Within this model:

- trust is not declared,
- authority is not inferred,
- and governance remains external, evidence-based, and accountable.

The report provides a foundation for:

- disciplined dataset formation,
- lifecycle-aware AI system development,
- and continued standards exploration

while preserving the core principle:

- Trust must be derived from evidence, not structure.

Annex B — External Artifact Onboarding and Adapter Ingestion (Informative)

B.1 Purpose

This annex defines an optional execution-layer mechanism for onboarding external or client-origin governance artifacts into the TR-008 signal-to-dataset transformation pipeline.

The purpose of onboarding and adapter ingestion is to:

- admit structured external artifacts into the execution environment,
- normalize heterogeneous input formats into the governed dataset contract,
- preserve provenance and authorization boundaries, and
- ensure all ingested artifacts enter the lifecycle under containment.

This annex is informative and non-normative.
It does not define governance authority, lifecycle rules, or authorization decisions.

B.2 Position Within the TR-008 Architecture

Adapter ingestion operates at the boundary between:

- external artifact sources, and
- the TR-008 signal-first execution pipeline.

It precedes:

signal capture
→ candidate formation
→ dataset packaging
→ pre-freeze
→ freeze

and provides a controlled entry point into the lifecycle defined in APOD-TR-007.

This layer does not replace dataset onboarding semantics defined in TR-007.
It implements them within the execution environment.

B.3 Accepted Artifact Sources

Adapter ingestion may admit artifacts from multiple upstream sources, including:

Access-Mode project outputs, including:

- ID_1 conceptual and learning artifacts used as informative reference material
- ID_2 operational reasoning records
- ID_3 inspection-oriented canonical artifacts

Working-PoD artifact sets, including:

- draft or populated module structures
- frozen ID_3 inspection snapshots

client or external governance data, including:

- structured governance records
- policy artifacts
- operational or analytical datasets

execution-derived artifacts, including:

- prior dataset packages
- runtime signal records
- lineage-linked dataset fragments

All such sources are treated as:

- non-authoritative and subject to governance constraints,
- context-bound, and
- subject to lifecycle containment.

B.4 Adapter Ingestion Process

Adapter ingestion performs the following operations:

1. **Source Classification**
    - identify artifact type
    - detect originating posture (ID_1 / ID_2 / ID_3 where applicable)
    - determine structural completeness

2. **Schema Mapping and Normalization**
    - convert input into canonical dataset-compatible structure
    - align fields to signal or dataset schema
    - enforce minimal structural requirements
3. **Provenance Preservation**
    - attach source references
    - retain canonical artifact identifiers where available
    - bind lineage anchors to input records
4. **Authorization Context Handling**
    - inherit authorization scope where defined
    - otherwise default to restricted containment posture
    - prevent implicit expansion of usage rights
5. **Lifecycle Initialization**
    - assign default lifecycle state:

      STATE_3 — provisional dataset layer

- attach lifecycle flags and routing-ready metadata

B.5 Default Routing Outcome

All ingested artifacts must default to:

STATE_3 (Runtime / Synthetic Dataset Layer)

This ensures alignment with the dataset containment principle defined in TR-007, where newly onboarded datasets remain provisional and not eligible for durable training use without explicit promotion.

No artifact may:

- bypass lifecycle entry,
- enter STATE_1 directly, or
- be treated as training-ready without validation and authorization.

B.6 Relationship to Signal Formation

Following ingestion:

- artifacts may be transformed into signal records, or
- signals may be extracted directly from structured inputs

All downstream processing must:

- conform to the signal schema defined in Section 5,
- preserve governance flags,
- and remain traceable to original sources.

Adapter ingestion does not produce final dataset artifacts.
It produces **normalized, lifecycle-contained inputs**.

B.7 Constraints and Non-Claims

Adapter ingestion must preserve the following invariants:

- Ingestion does not imply validation.
- Ingestion does not imply authorization.
- Ingestion does not imply correctness or completeness.
- Ingestion does not imply training eligibility.

The adapter layer must not:

- promote datasets across lifecycle states
- assign governance authority
- modify authorization scope without explicit governance input
- infer trust or compliance from structure

All ingested artifacts remain:

- provisional,
- non-authoritative, and
- subject to downstream validation and governance control.

B.8 Role Within the End-to-End System

Within the combined TR-006 → TR-007 → TR-008 model:

- **TR-006 (Access-Mode)** governs how artifacts are generated
- **TR-007** governs how datasets are defined, routed, and authorized
- **TR-008** governs how artifacts are operationally transformed

Adapter ingestion enables:

- user/client-generated artifacts to enter the execution pipeline
- normalization into governed dataset structures
- controlled progression into signal and dataset formation

This maintains separation between:

reasoning posture,
artifact state,
dataset lifecycle,
and governance authority.

B.9 Future Extension Path

This annex may be extended to support:

- automated schema adapters for client datasets
- sector-specific ingestion templates
- validation pre-checks prior to lifecycle routing
- integration with dataset registry and ingestion tooling

Future extensions must preserve:

- lifecycle containment,
- governance-first routing, and
- non-authoritative execution posture.

References

Siljeg, Anthony. *Q-Pathformer: Multi-state Machine Learning and LLM Training.* Alnotrea Labs Research Publication, December 2024. QPATH-ARCH-001. Originally published through the Q-Pathformer research series.

Policy of Developments — Access PoD series.

Vol.1 - *A Web of Sharing Trust-responsibility toward a Trustless Future*.

Vol.2A - APOD-TR-002 - Access-PoD Governance Framework
Vol.2B - APOD-TR-003 - Architecture Illustration
Vol.2C - APOD-TR-004 - Operational Onboarding Discipline
Vol.2D - APOD-TR-005 - Governance Stress Testing
Vol.2E - APOD-TR-006 - Execution Governance Extensions

Q-Pathformer – series.

APOD-TR-007 — Governed Dataset Onboarding and Lifecycle Routing

Q-Pathformer Enterprise v1.0 Document Set

APOD-TR-008 - Interaction-Role Signal Exchange and Execution Routing
APOD-TR-009 - CSC Session Model and Execution-Bound Assurance
APOD-TR-009-R - CSC Session Results and Evidence Profiles
QPATH-RUN-001 - Q-Pathformer Enterprise Run Results

GitHub Release

Q-Pathformer Enterprise v1.0 — TR-008 / TR-009 First Public Review Release

Repository:
https://github.com/qpathformer/qpathformer-enterprise-v1.0-release

Release tag:
https://github.com/qpathformer/qpathformer-enterprise-v1.0-release/releases/tag/v1.0-qpathformer-enterprise

---

© 2026 Anthony Siljeg

This document is provided for informational and educational purposes only. It is non-normative and does not define requirements, controls, or conformance criteria. No claim of compliance, certification, or deployment readiness may be derived from its use. Human judgment remains authoritative at all times.

All rights reserved.